NATIONAL UNIVERSITY OF MANAGEMENT

Faculty of Information Technology

Raspberry Pi Pico W

Getting Started with MQTT

MicroPython

Publish, Subscribe & Connect Embedded Devices to the IoT Cloud

Subject	Embedded Systems / IoT Communications
Level	Beginner to Intermediate
Duration	Approx. 90 Minutes
Language	MicroPython
Hardware	Raspberry Pi Pico W + BME280 Sensor / DHT22 Sensor
Prerequisite	Wi-Fi setup with MicroPython (Previous Lecture)
Primary Source	Random Nerd Tutorials — randomnerdtutorials.com
Date	April 2026

Contents

1. Lecture Overview & Learning Objectives 5
Learning Objectives 5
2. What Is MQTT? — Protocol Foundations 6
2.1 Why MQTT for IoT? 6
2.2 MQTT vs Other IoT Protocols 7
3. MQTT Core Concepts 7
3.1 Publish / Subscribe 7
3.2 Messages 8
3.3 Topics 8
3.4 The Broker 10
3.5 Quality of Service (QoS) 11
4. Setting Up the MQTT Broker 11
4.1 Option A — HiveMQ Cloud (Recommended) 12
HiveMQ Setup — Step by Step 12
4.2 Option B — Mosquitto on Raspberry Pi (Local) 14
5. Installing MQTT MicroPython Modules 15
5.1 Installation via Thonny IDE 15
Step 1 — Create the umqtt Directory 15
Step 2 — Create simple.py 16
Step 3 — Create robust.py 17
5.2 Alternative: mpremote mip install 17
5.3 Key umqtt.simple Methods 18
6. Project Setup — config.py & Wi-Fi Helper 19
6.1 config.py — Credentials File 19
6.2 Wi-Fi Connection Helper 20
6.3 Generating a Unique Client ID 20
7. Project A1 — Publishing MQTT Messages (BME280 Sensor) 21
7.1 Hardware Wiring — BME280 21
7.2 Complete Publisher Script 21
Importing Libraries 23
MQTT Topics 24
MQTT Details 24
BME280 Sensor 24
Initialize Wi-Fi 25
Connect to MQTT 25
Publish MQTT Messages 26
Publishing Sensor Readings 26

Testing the Code 27
7.3 Key Code Points 30
Payload Must Be Bytes 30
retain=True Explained 30
SSL for HiveMQ 30
7.4 Testing with MQTT Tools 30
7. Project A2 — Publishing MQTT Messages (DHT22 Sensor) 31
Changes Required 31
Hardware Connection 31
Revised MQTT Publisher Code (DHT22) 32
MQTT Topics Published 34
8. Project B — Subscribing to MQTT Topics (LED Control) 36
8.1 Hardware 36
8.2 Complete Subscriber Script 36
Subscribe to MQTT Topics 38
Callback Function 39
Connecting to Wi-Fi, MQTT Broker and Checking Messages 39
Testing the Code 40
8.3 Critical Subscription Rules 43
Rule 1 — Set Callback Before Subscribe 43
Rule 2 — Compare Bytes, Not Strings 43
check_msg() vs wait_msg() 43
8.4 Sending Test Commands 43
9. Advanced MQTT Features 44
9.1 umqtt.robust — Automatic Reconnection 44
9.2 Retained Messages 44
9.3 Last Will and Testament (LWT) 44
9.4 Multiple Topic Subscriptions & Wildcards 45
9.5 JSON Payloads 45
10. MQTT in Real IoT Systems 46
10.1 Integration Platforms 46
10.2 Node-RED Flow — Receive Pico W Sensor Data 46
10.3 Home Assistant YAML Configuration 46
10.4 Complete System Architecture 47
11. Featured Resource & Further Reading 48
11.1 Continuation Tutorials 48
11.2 Official References 48
12. Summary, Review Questions & Lab Exercises 49
12.1 Key Concepts Summary 49

12.2 Review Questions 49
A — MQTT Concepts 49
B — MicroPython Implementation 49
C — Advanced & Systems 50
12.3 Lab Exercises 50
Lab 1 — Publisher with Random Values (Beginner, 30 min) 50
Lab 2 — LED Subscriber (Beginner, 30 min) 50
Lab 3 — Bidirectional: Publish + Subscribe (Intermediate, 60 min) 50
Lab 4 — Node-RED Dashboard (Advanced, 90 min) 51
12.4 Quick Reference 51
simple.py 52
robust.py 57

1. Lecture Overview & Learning Objectives

This lecture introduces MQTT (Message Queuing Telemetry Transport) — the industry-standard lightweight messaging protocol for IoT. Students will master the publish/subscribe model, set up cloud and local MQTT brokers, install MicroPython umqtt modules, and build two real projects: a BME280 sensor data publisher and an LED control subscriber.

Learning Objectives

- Explain the MQTT publish/subscribe model and its IoT advantages over HTTP.
- Identify the four core concepts: Publish/Subscribe, Messages, Topics, and Broker.
- Set up a cloud MQTT broker (HiveMQ) and a local broker (Mosquitto).
- Install and configure umqtt.simple and umqtt.robust MicroPython modules.
- Create a modular config.py file for Wi-Fi and broker credentials.
- Publish BME280 sensor readings to MQTT topics every 10 seconds.
- Subscribe to MQTT topics and control an LED based on incoming messages.
- Understand QoS levels 0, 1, and 2 and their appropriate use cases.
- Apply MQTT in real IoT architectures with Node-RED and Home Assistant.

📌 Prerequisites

- Completion of: Raspberry Pi Pico W — Getting Started with Wi-Fi (MicroPython).
- MicroPython firmware flashed. Thonny IDE or VS Code with MicroPython extension installed.
- Hardware for full lab: BME280 sensor, LED, 220 Ohm resistor, breadboard, jumper wires.
- An active Wi-Fi network (2.4 GHz). Internet access required for HiveMQ cloud broker setup.

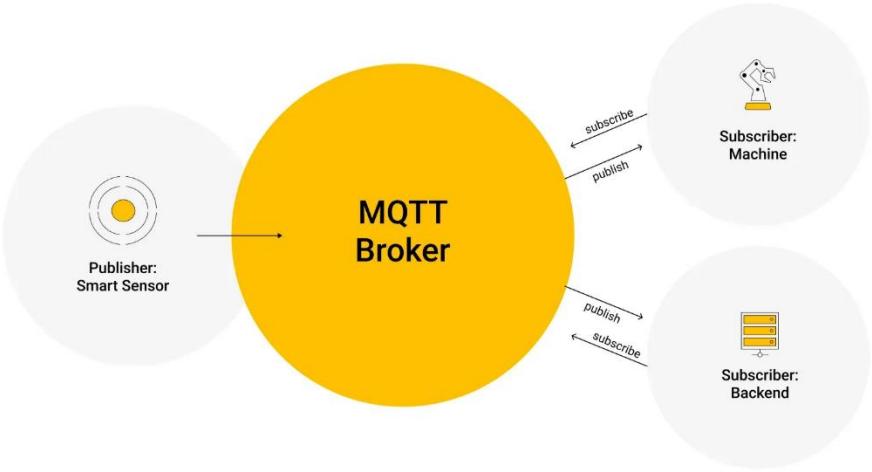
2. What Is MQTT? — Protocol Foundations

MQTT stands for Message Queuing Telemetry Transport. It is a lightweight, open-standard messaging protocol originally developed by IBM in 1999 for monitoring oil pipelines over satellite — environments where bandwidth was scarce and connections were unreliable. Today, MQTT is the dominant communication protocol for IoT, used by Amazon AWS IoT, Facebook Messenger, and millions of embedded devices worldwide.



2.1 Why MQTT for IoT?

Traditional HTTP is request-response: a client asks, a server answers, the connection ends. For IoT devices continuously reporting sensor readings or receiving commands, HTTP is inefficient. MQTT was designed specifically to overcome these limitations with a persistent connection and push-based messaging.



Feature	HTTP (Web)	MQTT (IoT)
Protocol model	Request-Response (Pull)	Publish-Subscribe (Push)
Connection	New connection per request	Persistent TCP connection
Bandwidth	High (~500 bytes minimum)	Very low (2-byte minimum packet)
Direction	Client initiates only	Any party can send at any time
Multiple receivers	One response per request	One message to many subscribers
Reliability	None built-in	QoS levels 0, 1, 2 available
Best for	Web pages, REST APIs	IoT sensors, home automation

> 💡 **Real-World MQTT Analogy**
>
> - Think of MQTT like a radio broadcast. Publishers are stations broadcasting on a frequency (topic).
> - Subscribers are radios tuned to that frequency — they receive the broadcast automatically.
> - The broker is the radio tower — it receives transmissions and retransmits to all listeners.
> - The publisher does not know or care who is listening. New listeners tune in without any changes to the station.

2.2 MQTT vs Other IoT Protocols

Protocol	Transport	Direction	Best For	Overhead
MQTT	TCP	Bidirectional pub/sub	IoT telemetry & control	Very low (2 bytes min)
HTTP/REST	TCP	Request-response	Web APIs, configuration	Medium (~500 bytes)
WebSocket	TCP	Full-duplex persistent	Real-time dashboards	Low (after handshake)
CoAP	UDP	Request-response	Ultra-constrained devices	Very low
AMQP	TCP	Message queue	Enterprise messaging	Medium

3. MQTT Core Concepts

MQTT is built on four fundamental concepts:

- Publish/Subscribe,
- Messages,
- Topics,
- and the Broker.

Mastering these is the foundation of all MQTT development.

3.1 Publish / Subscribe

Any MQTT client can **publish** messages to topics or **subscribe** to topics to receive messages. Publishers and subscribers never communicate directly — all messages pass through the broker. A Pico W publishing temperature data can simultaneously serve a dashboard, Home Assistant, a data logger, and a mobile app without any changes to its code.



- For example, **Device 1** publishes on a topic;
- **Device 2** is subscribed to the same topic that **Device 1** is publishing in;
- So, **Device 2** receives the message.

3.2 Messages

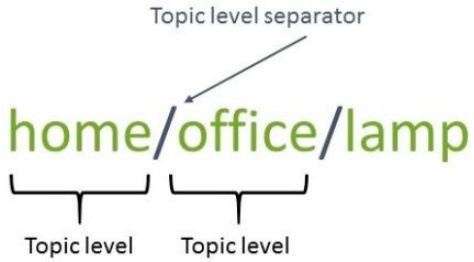
An MQTT message carries a payload (the data) and metadata. The payload is raw bytes and can contain any format: plain text, numbers, JSON, or binary data.

Payload	The actual data. Any byte string: b'25.3', b'{"temp":25.3}', b'on'
Topic	Routing address — determines which subscribers receive the message.
QoS	Delivery guarantee level: 0 (at most once), 1 (at least once), 2 (exactly once).
Retain	If True, broker stores last message. New subscribers receive it immediately.
DUP flag	Marks a retransmitted message (used internally under QoS 1 and 2).

3.3 Topics

Another important concept is the *topics*. Topics are the way you register interest in incoming messages or how you specify where you want to publish the message.

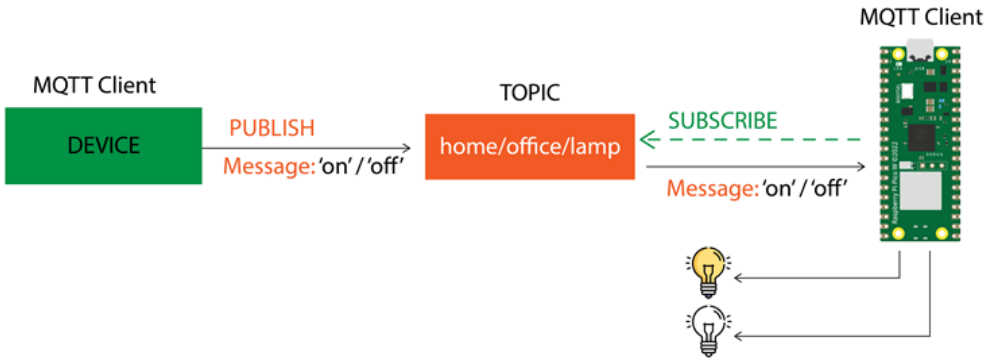
Topics are hierarchical strings using / as a separator. They represent the logical structure of your IoT system.



Note: topics are case-sensitive, which makes the following topics different.



If you would like to turn on a lamp in your home office using MQTT you can imagine the following scenario:



1. A device publishes on and off messages on the home/office/lamp topic.
2. You have a device that controls a lamp (it can be your Raspberry Pi Pico, or any other board or device). The Pico, that controls your lamp, is subscribed to that same topic: home/office/lamp.
3. So, when a new message is published on that topic, the Pico receives the on or off messages and turns the lamp on or off.

The device that is publishing the messages can be another microcontroller board, or a Home Automation controller platform with MQTT support like Node-RED, Home Assistant, Adafruit IO, Domoticz, or OpenHAB, for example.

```
# Pico W topic hierarchy examples:
pico/temperature          # Temperature reading
pico/humidity             # Humidity reading
pico/pressure             # Pressure reading
pico/led                  # LED control (subscribe)
pico/status               # Online/offline status

# Home automation topic hierarchy:
home/office/temperature
home/office/lamp          # Control lamp in office
home/bedroom/temperature

# Wildcard subscriptions:
pico/#                    # All pico/ topics (multi-level wildcard)
home/+/temperature        # Temperature in ANY room (single-level wildcard)
```

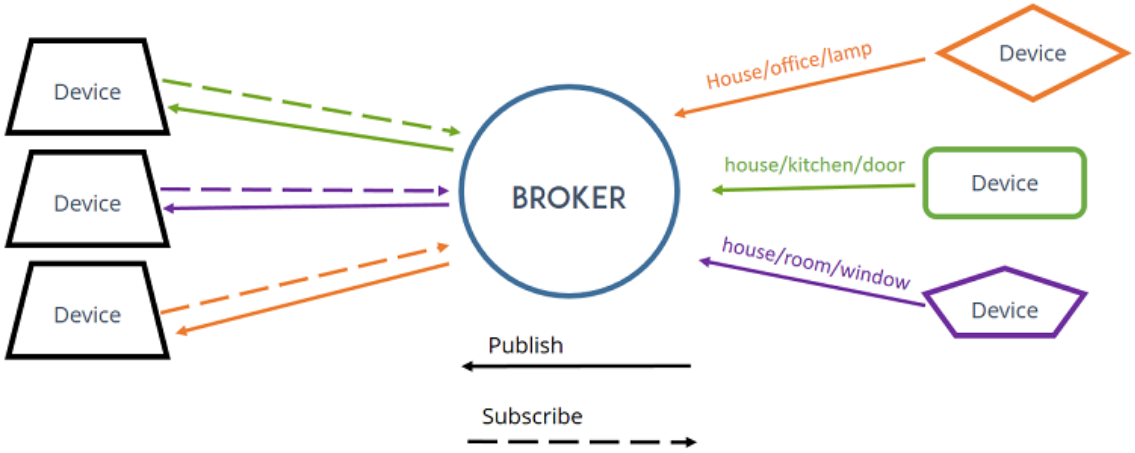
⚠ Topics Are Case Sensitive

- pico/temperature and pico/Temperature and Pico/Temperature are three different topics.
- Always use consistent lowercase naming in your topic hierarchy.
- A subscriber to pico/temperature will NOT receive messages published to pico/Temperature.

3.4 The Broker

Finally, another important concept is the *broker*.

The MQTT broker is the central hub that receives all published messages, filters by topic, and forwards each message to all subscribed clients. It also handles authentication, session state, retained messages, and Last Will.



There are several brokers you can use. For example:

1. **Cloud MQTT Broker solutions**: you can use commercial MQTT broker solutions like HiveMQ, for example. You don't need to set up anything. You just need to create an account and you're ready to go (this is the one we'll use in this tutorial).
2. **Local MQTT broker**: you can install an MQTT broker locally on your computer or on your Raspberry Pi. The **Mosquitto** MQTT broker hosted on a Raspberry Pi is widely used in many hobbyist projects and it's also the solution we use more often.
3. **Cloud MQTT Broker**: as an alternative to the previous solution, you can also install MQTT broker on your own cloud server.

In summary:

- MQTT is a communication protocol very useful in Internet of Things projects;
- In MQTT, devices can publish messages on specific topics and can be subscribed to topics to receive messages;
- You need a broker when using MQTT. It receives all the messages and sends them to the subscribed devices.

Core function	Receive, filter, and forward messages between all clients.
Authentication	Username/password verification. TLS/SSL for encrypted connections.
Retained messages	Stores the last message per topic for new subscribers.
Last Will (LWT)	Sends a predefined message when a client disconnects unexpectedly.
Session state	Remembers persistent subscriptions across client reconnections.

3.5 Quality of Service (QoS)

QoS	Name	Guarantee	Use Case	Cost
0	At Most Once	No guarantee. May be lost.	High-frequency sensors where occasional loss is OK.	Zero overhead
1	At Least Once	Delivered, possibly multiple times.	Commands where delivery matters but duplicates are tolerable.	ACK packet
2	Exactly Once	Delivered exactly once. No duplicates.	Financial, critical actuators (door lock, alarm).	4-packet handshake

4. Setting Up the MQTT Broker

To use MQTT, you need an MQTT broker. The broker receives all MQTT messages and sends them to all subscribed clients.

MQTT Broker Solutions
There are many MQTT broker solutions you can use. Here we'll describe the ones we're more familiar with:

1. Mosquitto MQTT broker installed on a Raspberry Pi: this is an open-source MQTT broker that you can install locally on your Raspberry Pi (the Raspberry Pi computer, not the Pico board). We've been using this option a lot and it always worked well for us.
2. Mosquitto MQTT broker installed on a cloud server: a great alternative if you want your broker to be accessible worldwide.
3. HiveMQ Broker: this is a cloud MQTT broker service. You just need to create an account and choose a plan. They provide a free plan that is suitable for most IoT hobbyist projects.

Broker	Type	Setup Effort	Internet Required	Best For
HiveMQ Cloud	Cloud (Managed)	None — just register	Yes	Quick start, learning, cloud IoT
Mosquitto on RPi	Local (Self-hosted)	Medium	No	Offline use, home automation lab
Mosquitto on VPS	Cloud (Self-hosted)	High	Yes (always-on)	Production, remote worldwide access
Adafruit IO	Cloud (Managed)	Low	Yes	Beginners, built-in dashboard

For simplicity, in this guide we'll use HiveMQ because you don't need to install or configure anything like in the Mosquitto MQTT broker. You just need to create an account, set up a cluster and you're ready to go. Alternatively, if you want to use Mosquitto broker or any other broker of your choice, don't worry. The code will be compatible with any broker as long as you fill in the right broker details.

Regardless of the MQTT broker you use, you need to have the MQTT URL, username, and password before proceeding to the following sections.

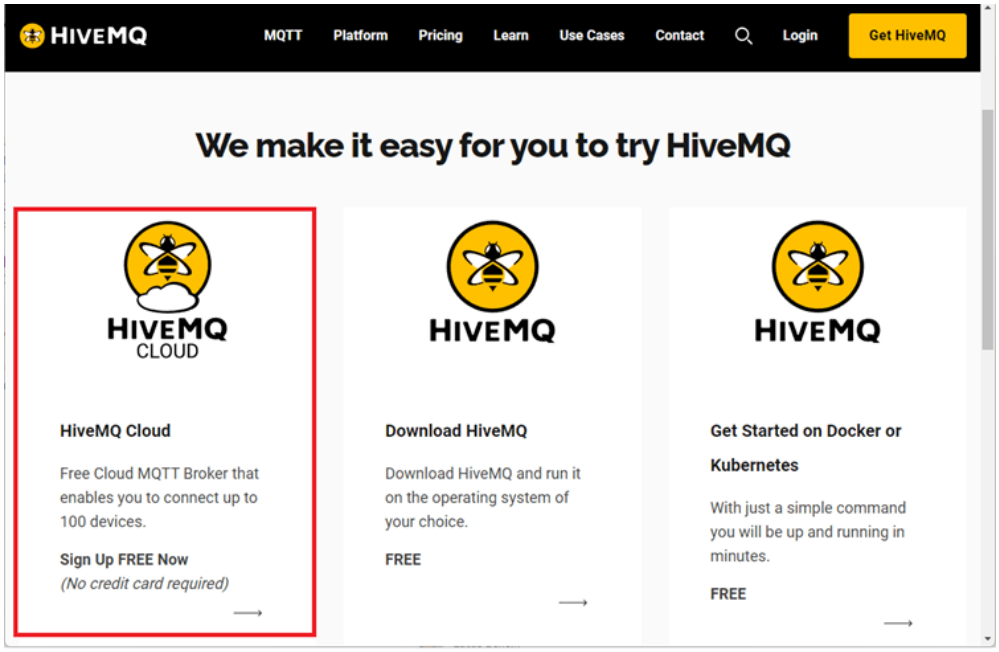
4.1 Option A — HiveMQ Cloud (Recommended)

In this section, we'll show you how to set up your HiveMQ MQTT broker.

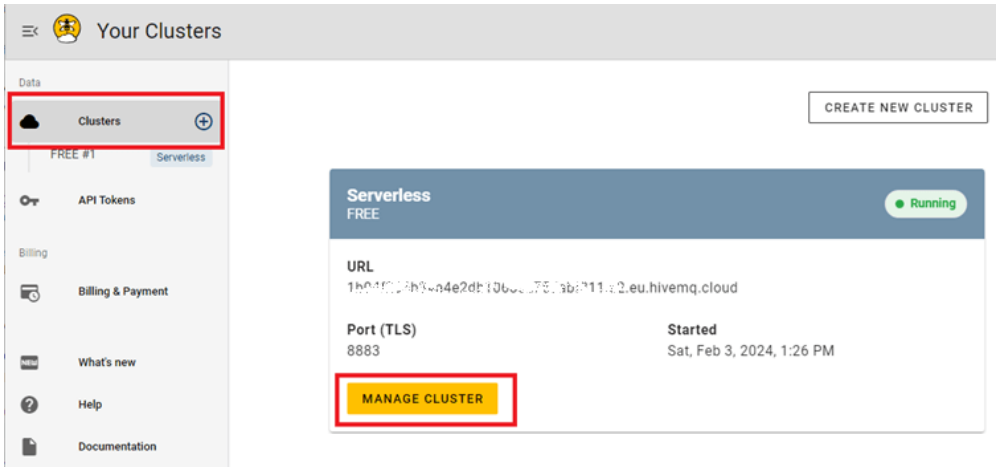
HiveMQ provides a free cloud MQTT broker with no software installation required. Simply create an account and you are ready.

HiveMQ Setup — Step by Step

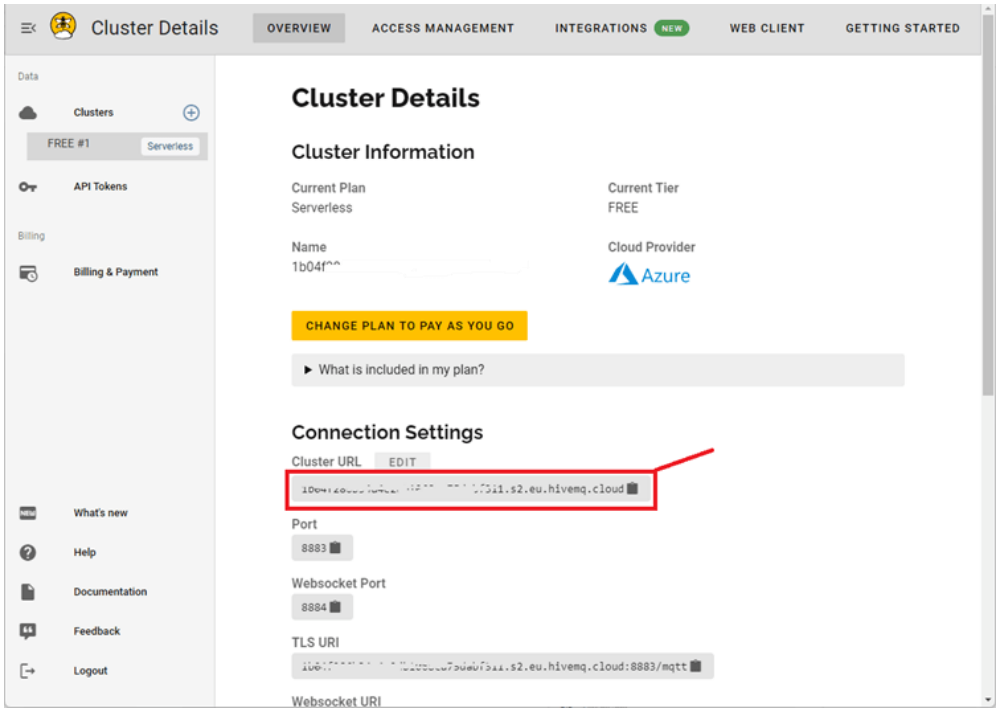
1. Go to hivemq.com and click Start free.
2. Select the HiveMQ Cloud **free plan**.



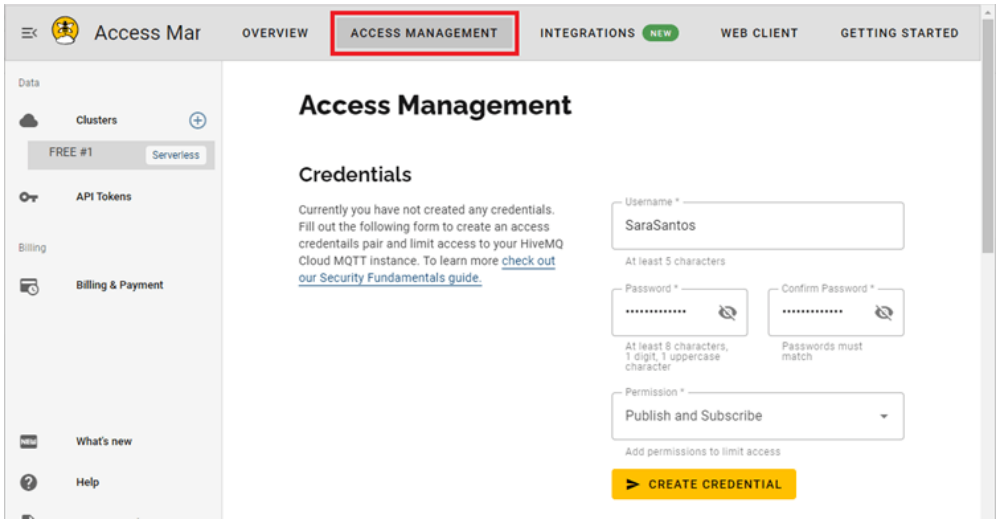
3. Create an account or log in. Complete your profile.
4. A default cluster is created automatically. Click **Manage Cluster**.



5. Copy and save the Cluster URL (e.g., abc123.s1.eu.hivemq.cloud). This is your mqtt_server.



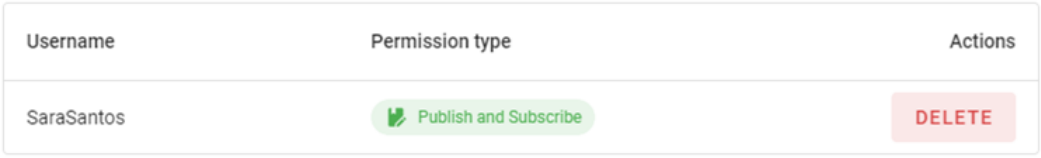
6. Click the **Access Management tab**.
7. Enter a **username** and **password**. Set permission to *Publish* and *Subscribe*.
8. Finally, Click **Create Credential**.



Record These Details After Setup

```
mqtt_server   = b'YOUR_CLUSTER.hivemq.cloud'
mqtt_port     = 8883   (TLS/SSL port — HiveMQ requires SSL)
mqtt_username = b'your_username'
mqtt_password = b'your_password'
```

If your credentials are successfully created, your MQTT broker is set up.



Make sure you have the following information before proceeding to the next section:

- MQTT Server:
  - Cluster URL (for HiveMQ)
  - MQTT broker IP address or URL (if you're using a different MQTT Broker)
- MQTT username
- MQTT password

4.2 Option B — Mosquitto on Raspberry Pi (Local)

```
# Install Mosquitto on Raspberry Pi (Debian/Ubuntu):
sudo apt update && sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable mosquitto && sudo systemctl start mosquitto

# Edit config to allow remote connections:
sudo nano /etc/mosquitto/mosquitto.conf
# Add:  listener 1883
# Add:  allow_anonymous true   (for testing only!)

# Restart and get Raspberry Pi IP:
sudo systemctl restart mosquitto
hostname -I   # Use this IP as mqtt_server in config.py

# Test from another terminal:
mosquitto_sub -h localhost -t 'pico/#'
mosquitto_pub -h localhost -t 'pico/test' -m 'hello'
```

> ⚠ **Security Note**
> allow_anonymous true is acceptable on a private local network for learning.
> NEVER use allow_anonymous true on a public-facing broker.
> For production: always use username/password authentication AND TLS encryption.

5. Installing MQTT MicroPython Modules

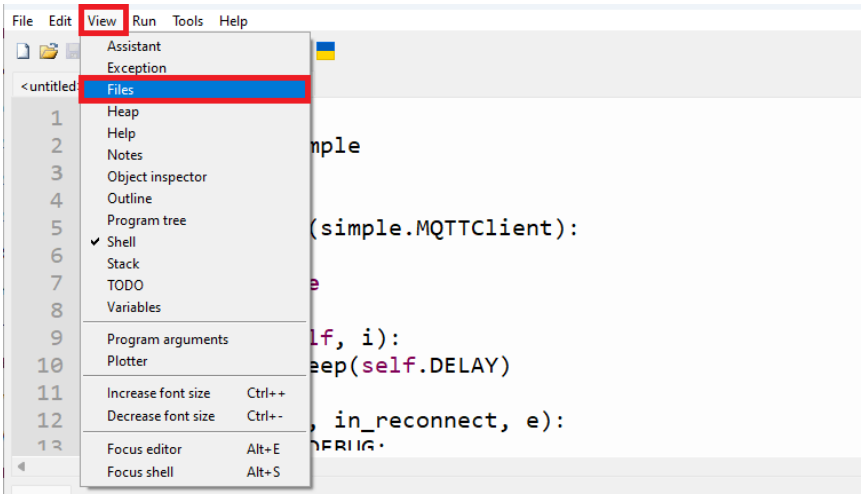
MicroPython does not include an MQTT client in its standard library. We install two modules: umqtt.simple (core MQTT client) and umqtt.robust (extends simple with automatic reconnection). Both are lightweight and designed for microcontrollers.

Module	File	Purpose	When to Use
umqtt.simple	umqtt/simple.py	Core MQTT client: connect, publish, subscribe, check.	Basic projects, stable connections.
umqtt.robust	umqtt/robust.py	Extends simple with auto-reconnect on network failure.	Production, unstable networks.

5.1 Installation via Thonny IDE

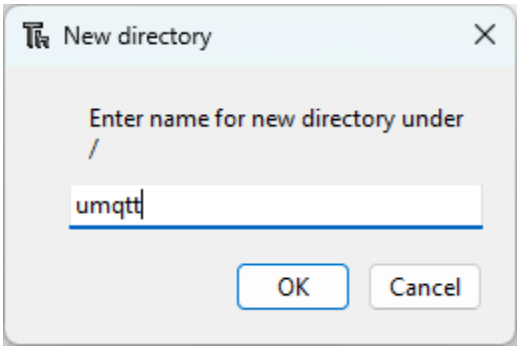
Step 1 — Create the umqtt Directory

1. Connect Raspberry Pi Pico W via USB and open Thonny IDE.
2. Go to View > Files to show the filesystem panel.

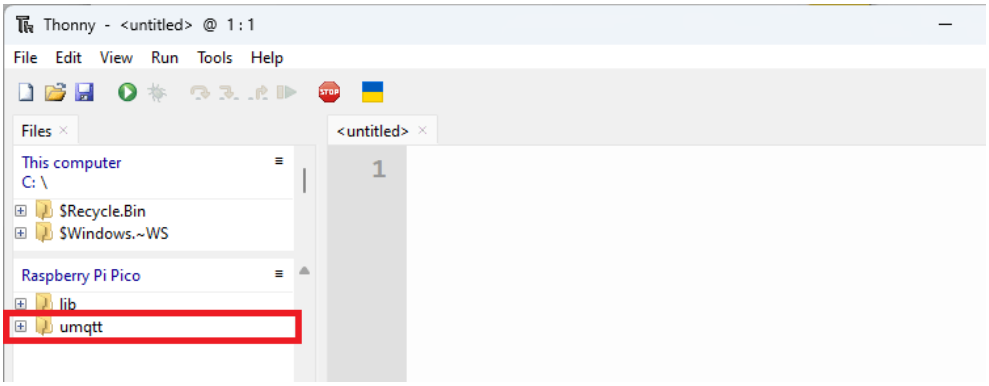


A new sidebar will show up with all the files on the Raspberry Pi Pico filesystem.

3. In the Pico W filesystem panel, right-click and select **New directory**...
4. This new directory should be called ***umqtt***. Click **Ok**.

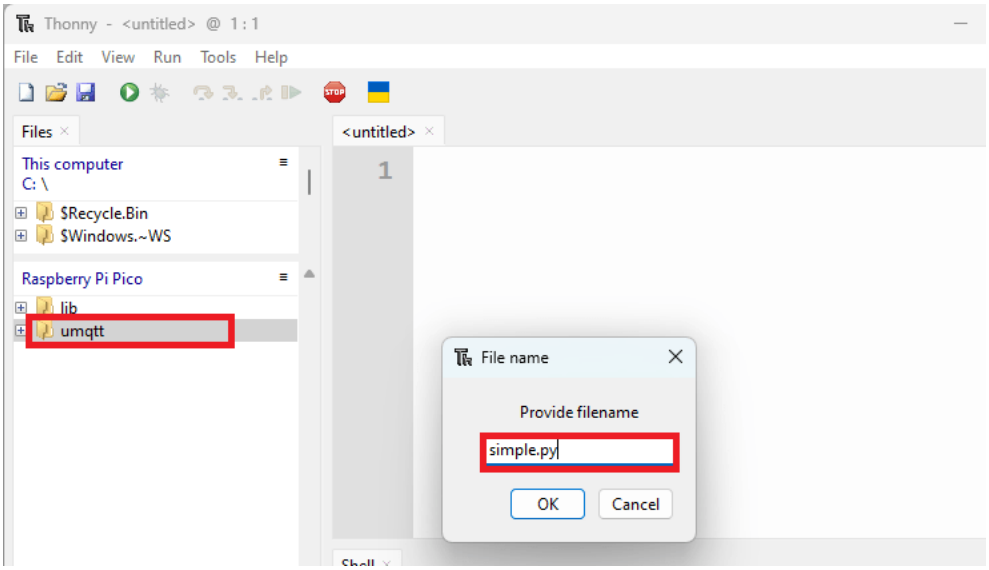


The new directory will show up on the left sidebar.



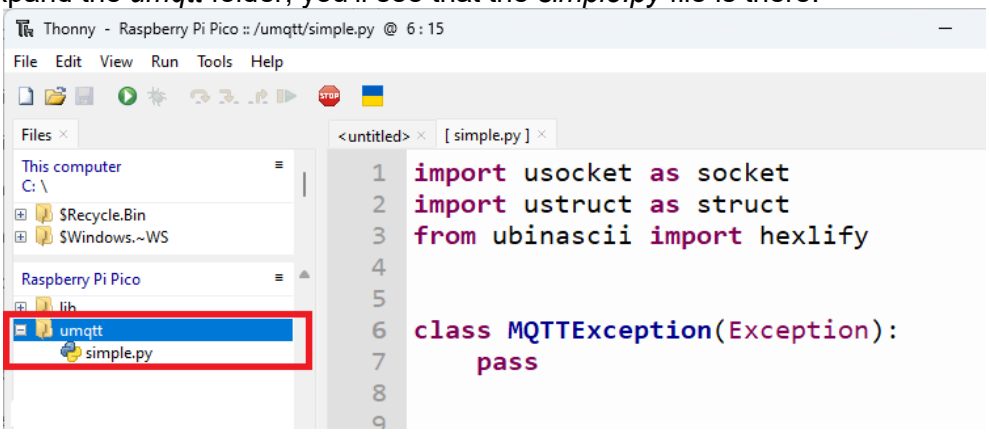
Step 2 — Create simple.py

1. Right-click the **umqtt** folder and select **New file**... Name it **simple.py**.



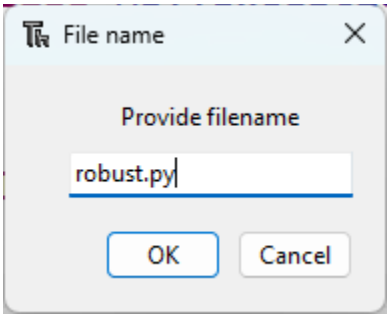
2. Download **simple.py** from: github.com/RuiSantosdotme/Random-Nerd-Tutorials/blob/master/Projects/Raspberry-Pi-Pico/MicroPython/mqtt/umqtt/simple.py
3. Copy the entire code into the file and press **Ctrl+S** to **save**.

Now, if you expand the *umqtt* folder, you'll see that the *simple.py* file is there.

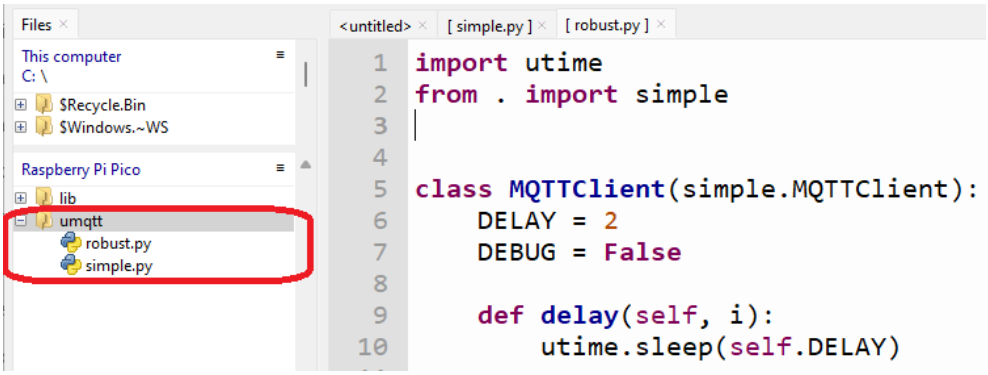


Step 3 — Create robust.py

1. Right-click the **umqtt** folder and select **New file**... Name it **robust.py**.



2. Download **robust.py** from the same GitHub repository directory.
3. Copy the code into the file and save.
4. After copying the code to the *robust.py* file, save the code. At this moment, you should have the ***umqtt*** folder with the ***simple***.*py* and ***robust***.*py* files inside.



The modules required for MQTT were successfully uploaded to the Raspberry Pi Pico.

```
# Expected Pico W filesystem after installation:
/ (root)
├── umqtt/
│   ├── simple.py      # Core MQTTClient class
│   └── robust.py      # Auto-reconnect wrapper
└── config.py          # Created in next section
```

5.2 Alternative: mpremote mip install

```
# For MicroPython v1.20+, install from terminal:
pip install mpremote
mpremote mip install umqtt.simple
mpremote mip install umqtt.robust
```

5.3 Key umqtt.simple Methods

MQTTClient(id, server, port, user, password, ssl)	Constructor. Creates MQTT client with all connection parameters.
.connect(clean_session=True)	Connect to broker. clean_session=True = no persistent session.
.publish(topic, msg, retain, qos)	Publish msg to topic. Both must be bytes (b'...' prefix).
.set_callback(func)	Register function called on each incoming message. MUST be set before subscribe().
.subscribe(topic, qos)	Subscribe to a topic. set_callback must be called first.
.check_msg()	Non-blocking: check for messages, return immediately if none.
.wait_msg()	Blocking: halt until a message arrives.
.set_last_will(topic, msg, retain, qos)	Register LWT message. MUST be called before connect().
.ping()	Send PINGREQ to keep connection alive.
.disconnect()	Gracefully close the MQTT connection.

6. Project Setup — config.py & Wi-Fi Helper

Before writing application code, we establish a clean project structure.

A **config.py** file stores all credentials separately, making it easy to update without touching application logic.

We'll create a configuration file to save the **SSID**, **password**, and your **MQTT broker** details: **URL**, **username**, and **password**.

6.1 config.py — Credentials File

Create a new file called ***config.py*** on Thonny IDE and copy the following code:

```
# config.py — Save to ROOT of Pico W filesystem (NOT inside umqtt/)

# Wi-Fi Settings (regular strings, no b prefix)
wifi_ssid     = 'REPLACE_WITH_YOUR_SSID'
wifi_password = 'REPLACE_WITH_YOUR_PASSWORD'

# MQTT Settings — MUST be bytes (b prefix required!)

# For HiveMQ Cloud (SSL, port 8883):
mqtt_server   = b'YOUR_CLUSTER.hivemq.cloud'
mqtt_port     = 8883
mqtt_username = b'YOUR_USERNAME'
mqtt_password = b'YOUR_PASSWORD'

# For local Mosquitto (no SSL, port 1883):
# mqtt_server   = b'192.168.1.79'
# mqtt_port     = 1883
# mqtt_username = b''
# mqtt_password = b''
```

> ⚠ **Bytes vs Strings — Critical Difference**
>
> MQTT server, username, and password MUST be bytes objects (b'...' prefix).
> The umqtt library passes them directly to the TCP socket as binary data.
> Wi-Fi SSID and password are normal strings (no b prefix) — they go to the network module.
> Common mistake: TypeError if you forget the b prefix on MQTT credentials.

Then, go to **File** > **Save as…** and select **Raspberry Pi Pico**. Save the file as *config.py* (overwrite any existing files with the same name). This file should be saved on the root of the Raspberry Pi Pico and **not** inside the *umqtt* folder.

6.2 Wi-Fi Connection Helper

```
# Wi-Fi helper — include in each project script
import network
from time import sleep
import config

# Init Wi-Fi Interface
def initialize_wifi(ssid, password):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    # Connect to the network
    wlan.connect(ssid, password)

    # Wait for Wi-Fi connection
    connection_timeout = 10
    while connection_timeout > 0:
        if wlan.status() >= 3:
            break
        connection_timeout -= 1
        print('Waiting for Wi-Fi connection...')
        sleep(1)

    # Check if connection is successful
    if wlan.status() != 3:
        return False
    else:
        print('Connection successful!')
        network_info = wlan.ifconfig()
        print('IP address:', network_info[0])
        return True
```

6.3 Generating a Unique Client ID

Every MQTT client must have a unique Client ID. Duplicate IDs cause the broker to disconnect the earlier client. Use the hardware MAC address to generate a guaranteed-unique ID.

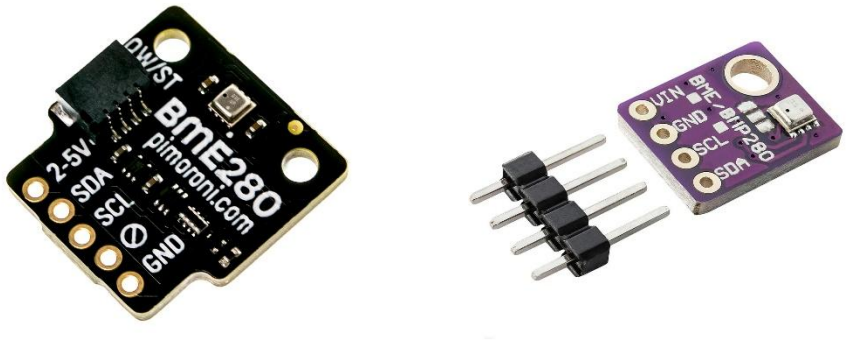
```
from ubinascii import hexlify
import machine


# Generate unique client ID from Pico W hardware ID:
client_id = hexlify(machine.unique_id())
# Example: b'e66130ab1234' (hex of hardware unique ID)


# Or with a readable prefix:
mac = hexlify(machine.unique_id()).decode()
client_id = ('picow_' + mac[:6]).encode()
# Example: b'picow_e66130'
```

7. Project A1 — Publishing MQTT Messages (BME280 Sensor)

The Pico W reads temperature, humidity, and pressure from a BME280 sensor and publishes each reading to a dedicated MQTT topic every 10 seconds. Any subscribed client receives these readings in real time.



7.1 Hardware Wiring — BME280

BME280 SDA	GPIO 4 (Pin 6) on Pico W
BME280 SCL	GPIO 5 (Pin 7) on Pico W
BME280 VCC	3.3V (Pin 36) on Pico W
BME280 GND	GND (Pin 38) on Pico W

> 💡 **No BME280? Use Random Values for Testing**
>
> ```
> import random
> temperature = str(round(random.uniform(20.0, 35.0), 2)) + 'C'
> humidity    = str(round(random.uniform(40.0, 80.0), 2)) + '%'
> pressure    = str(round(random.uniform(990.0, 1020.0), 2)) + 'hPa'
> ```
>
> This lets you test all MQTT publish logic without specific hardware.

7.2 Complete Publisher Script

The following code gets sensor data from the BME280 sensor and publishes the readings on different MQTT topics.

```
# Raspberry Pi Pico W – MQTT Publisher (BME280 Sensor Data)
# Source: Random Nerd Tutorials
# https://randomnerdtutorials.com/raspberry-pi-pico-w-mqtt-micropython/

from machine import Pin, I2C
from time import sleep
import network
from umqtt.simple import MQTTClient
```

```
from ubinascii import hexlify
import machine, config, BME280

# MQTT Topic constants — bytes required
TOPIC_TEMP     = b'pico/temperature'
TOPIC_HUMIDITY = b'pico/humidity'
TOPIC_PRESSURE = b'pico/pressure'

def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        wlan.connect(config.wifi_ssid, config.wifi_password)
        timeout = 10
        while not wlan.isconnected() and timeout > 0:
            sleep(1); timeout -= 1
    if wlan.isconnected():
        print('Wi-Fi connected. IP:', wlan.ifconfig()[0]); return True
    return False

def connect_mqtt():
    client_id = hexlify(machine.unique_id())
    client = MQTTClient(
        client_id  = client_id,
        server     = config.mqtt_server,
        port       = config.mqtt_port,
        user       = config.mqtt_username,
        password   = config.mqtt_password,
        keepalive  = 60,
        ssl        = True,
        ssl_params = {'server_hostname': config.mqtt_server}
    )
    client.connect()
    print('Connected to broker:', config.mqtt_server)
    return client

# Initialise I2C and BME280
i2c = I2C(0, sda=Pin(4), scl=Pin(5), freq=400000)
bme = BME280.BME280(i2c=i2c)

if not connect_wifi():
    raise RuntimeError('Wi-Fi connection failed')
client = connect_mqtt()
```

```
while True:
    try:
        # Read sensor (BME280 library returns strings like '25.23C')
        temperature = bme.temperature
        humidity    = bme.humidity
        pressure    = bme.pressure
        print(f'Temp:{temperature}  Hum:{humidity}  Press:{pressure}')

        # Publish — payload must be bytes (.encode() converts string to bytes)
        client.publish(TOPIC_TEMP,     temperature.encode(), retain=True)
        client.publish(TOPIC_HUMIDITY, humidity.encode(),    retain=True)
        client.publish(TOPIC_PRESSURE, pressure.encode(),    retain=True)
        print('Published successfully')

    except Exception as e:
        print('Error:', e)
        try: client = connect_mqtt()
        except Exception as re: print('Reconnect failed:', re)

    sleep(10)
```

We use the following MQTT topics to publish data:

- For the temperature — pico/temperature
- For the humidity — pico/humidity
- For the pressure — pico/pressure

And the workflow to publish data is the following:

1. Connect the Pico to the internet;
2. Connect to the MQTT broker;
3. After connecting to the broker, we can continuously publish MQTT messages.

Let's take a look at how the code works.

Importing Libraries

First, we need to import the required libraries. We import the `Pin` and `I2C` from the `machine` module, and the `BME280` to interface with the BME280 sensor. The `network` module to connect to Wi-Fi and the `MQTTClient` class from umqtt.simple to use MQTT functions.

```
from machine import Pin, I2C
from time import sleep
import network
from umqtt.simple import MQTTClient
import config
import BME280
```

MQTT Topics

On the following variables we save the topics where we want to publish our messages (sensor readings).

```
# Constants for MQTT Topics
MQTT_TOPIC_TEMPERATURE = 'pico/temperature'
MQTT_TOPIC_PRESSURE = 'pico/pressure'
MQTT_TOPIC_HUMIDITY = 'pico/humidity'
```

MQTT Details

In the following lines we set up the MQTT parameters to connect to our MQTT broker. We import the server URL, username, and password from the ***config.py*** file.

```
# MQTT Parameters
MQTT_SERVER = config.mqtt_server
MQTT_PORT = 0
MQTT_USER = config.mqtt_username
MQTT_PASSWORD = config.mqtt_password
MQTT_CLIENT_ID = b"raspberrypi_picow"
MQTT_KEEPALIVE = 7200
MQTT_SSL = True
MQTT_SSL_PARAMS = {'server_hostname': MQTT_SERVER}
```

The `MQTT_CLIENT_ID` should be a unique ID to identify the MQTT client. You can give it wherever name you want, but it needs to be unique among the MQTT clients connected to your broker. In our case, it's `raspberrypi_picow`.

```
MQTT_CLIENT_ID = b"raspberrypi_picow"
```

If you're using a local Mosquitto MQTT broker, you should set the `MQTT_SSL` parameter to `False`.

```
MQTT_SSL = False
```

BME280 Sensor

In the following lines, we initialize the BME280 sensor and create a function called `get_sensor_readings()` that gets data from the BME280 and returns the temperature, humidity, and pressure.

```
# Initialize I2C communication
i2c = I2C(id=0, scl=Pin(5), sda=Pin(4), freq=10000)

# Initialize BME280 sensor
bme = BME280.BME280(i2c=i2c, addr=0x76)

def get_sensor_readings():
    temp = bme.temperature[:-1]
    hum = bme.humidity[:-1]
    pres = bme.pressure[:-3]
    return temp, hum, pres
```

Initialize Wi-Fi

The `initialize_wifi()` function connects the Raspberry Pi Pico to a network. You need to initialize Wi-Fi to connect to the MQTT broker and exchange messages.

```
def initialize_wifi(ssid, password):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    # Connect to the network
    wlan.connect(ssid, password)

    # Wait for Wi-Fi connection
    connection_timeout = 10
    while connection_timeout > 0:
        if wlan.status() >= 3:
            break
        connection_timeout -= 1
        print('Waiting for Wi-Fi connection...')
        sleep(1)

    # Check if connection is successful
    if wlan.status() != 3:
        return False
    else:
        print('Connection successful!')
        network_info = wlan.ifconfig()
        print('IP address:', network_info[0])
        return True
```

Connect to MQTT

The `connect_mqtt()` function connects to the MQTT broker using the details about the MQTT broker you've set up previously.

```
def connect_mqtt():
    try:
        client = MQTTClient(client_id=MQTT_CLIENT_ID,
                            server=MQTT_SERVER,
                            port=MQTT_PORT,
                            user=MQTT_USER,
                            password=MQTT_PASSWORD,
                            keepalive=MQTT_KEEPALIVE,
                            ssl=MQTT_SSL,
                            ssl_params=MQTT_SSL_PARAMS)
        client.connect()
        return client
    except Exception as e:
        print('Error connecting to MQTT:', e)
        raise  # Re-raise the exception to see the full traceback
```

Publish MQTT Messages

The `publish_mqtt()` function publishes a message on a topic. Pass as argument the topic and the message you want to send.

```
def publish_mqtt(topic, value):
    client.publish(topic, value)
    print(topic)
    print(value)
    print("Publish Done")
```

Publishing Sensor Readings

Now that we have all functions and variables defined, we can finally connect to the internet and to the broker to start publishing messages.

First, we try to connect to Wi-Fi using the SSID and password stored on the ***config.py*** file.

```
try:
    if not initialize_wifi(config.wifi_ssid, config.wifi_password):
        print('Error connecting to the network... exiting program')
```

If we succeed in connecting to Wi-Fi, we can connect to the MQTT broker.

```
else:
    # Connect to MQTT broker, start MQTT client
    client = connect_mqtt()
```

After connecting, we get new sensor data and save it on the temperature, humidity, and pressure variables.

```
# Read sensor data
temperature, humidity, pressure = get_sensor_readings()
```

Finally, we use the `publish_mqtt()` function to publish the readings on their specific topics. The message must be a string. So, we need to convert the sensor data using the `str()` function.

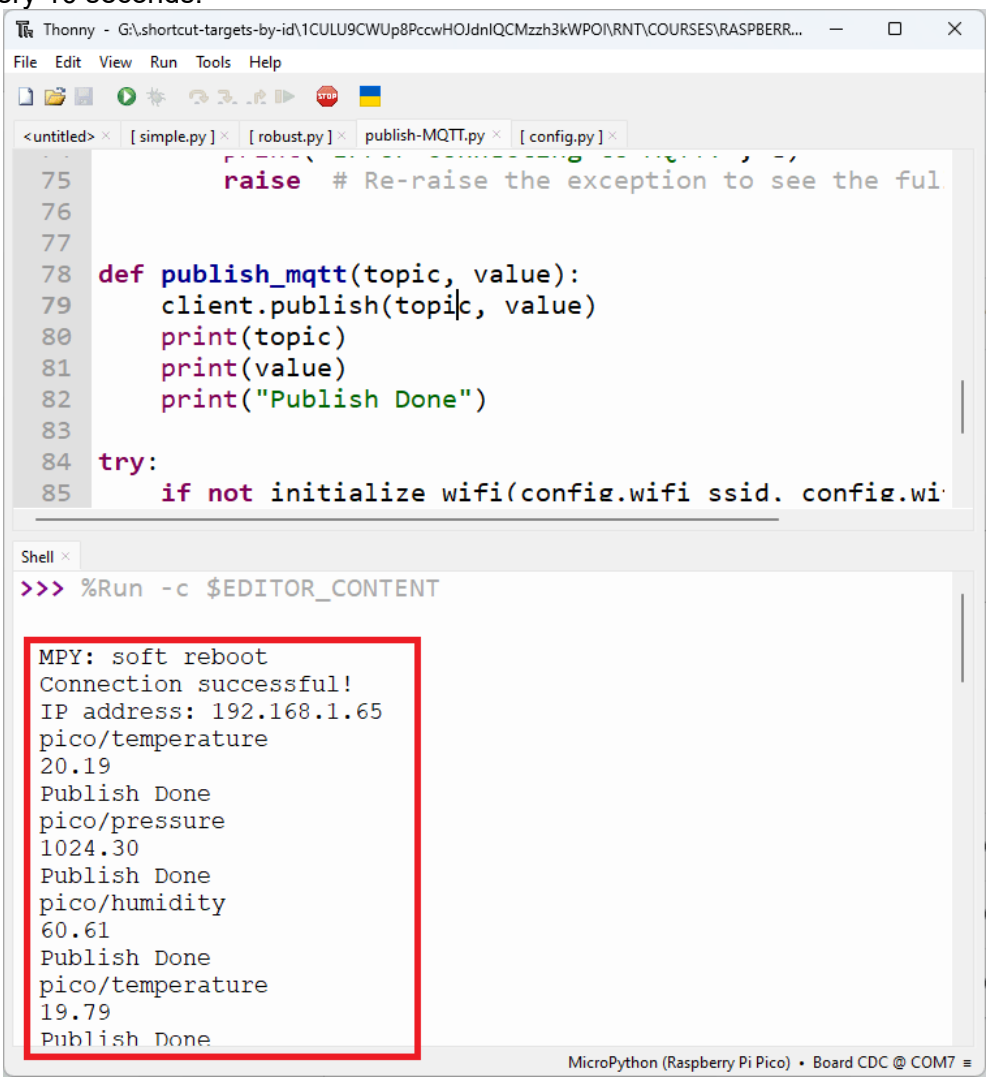
```
# Publish as MQTT payload
publish_mqtt(MQTT_TOPIC_TEMPERATURE, str(temperature))
publish_mqtt(MQTT_TOPIC_PRESSURE, str(pressure))
publish_mqtt(MQTT_TOPIC_HUMIDITY, str(humidity))
```

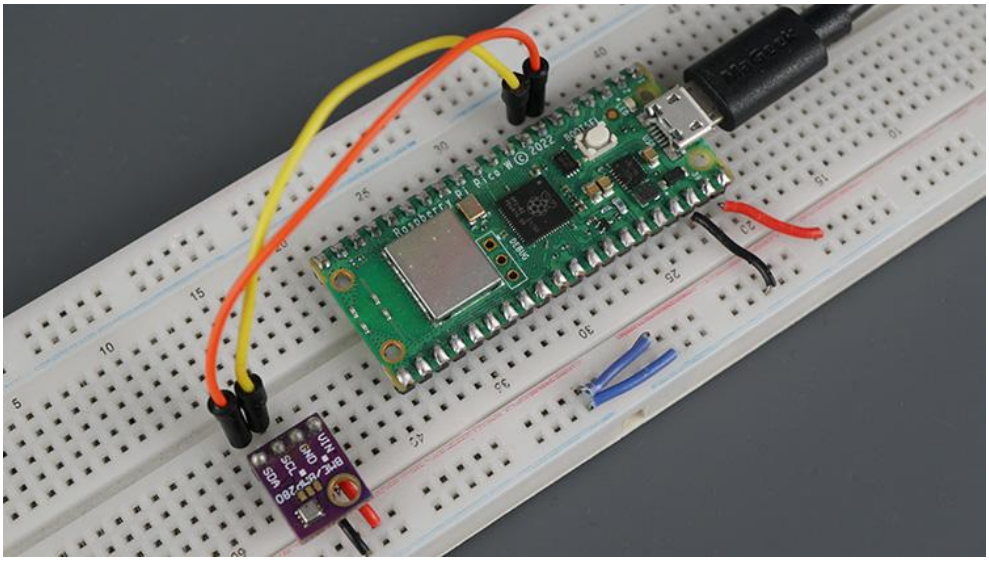
We publish new readings every 10 seconds.

```
sleep(10)
```

Testing the Code

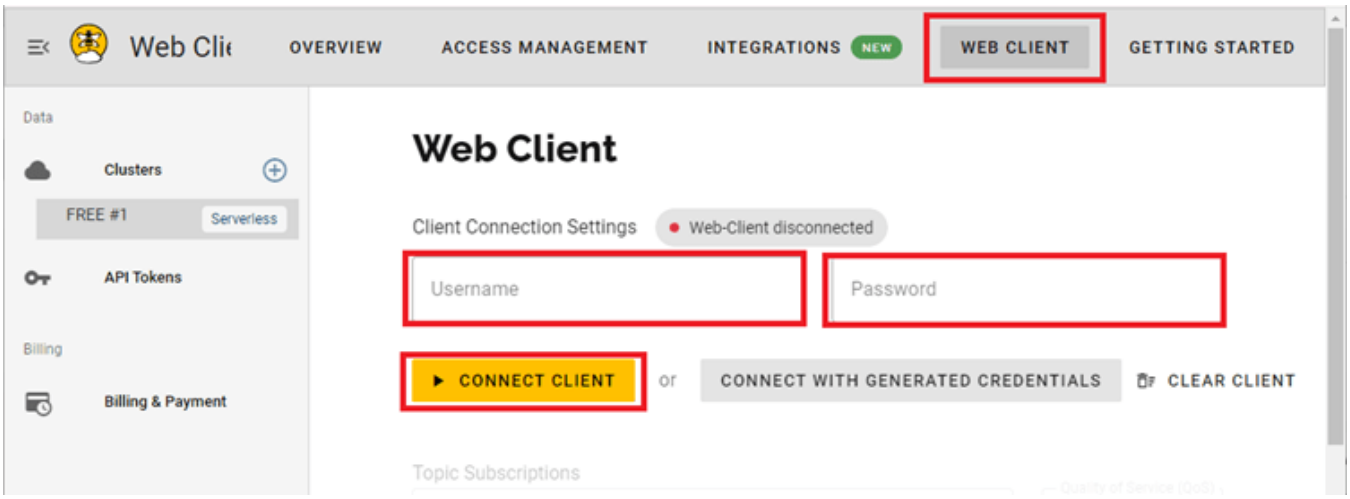
Run the previous code on your Raspberry Pi Pico. It will connect to the internet and start publishing messages every 10 seconds.



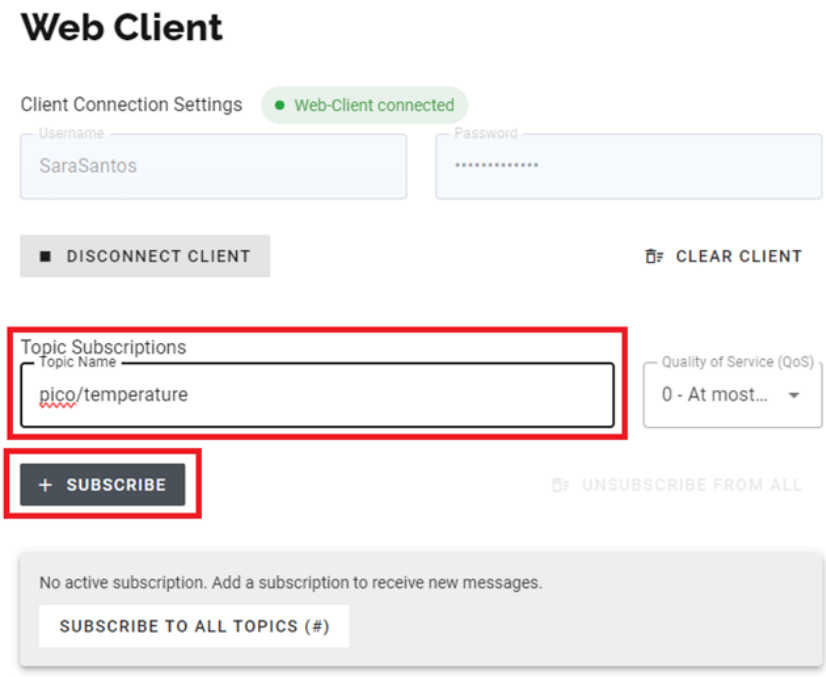


Now, we can check if we can receive the messages on an MQTT Client subscribed to those topics. HiveMQ provides a Web Client interface that allows you to subscribe and publish to topics for testing purposes.

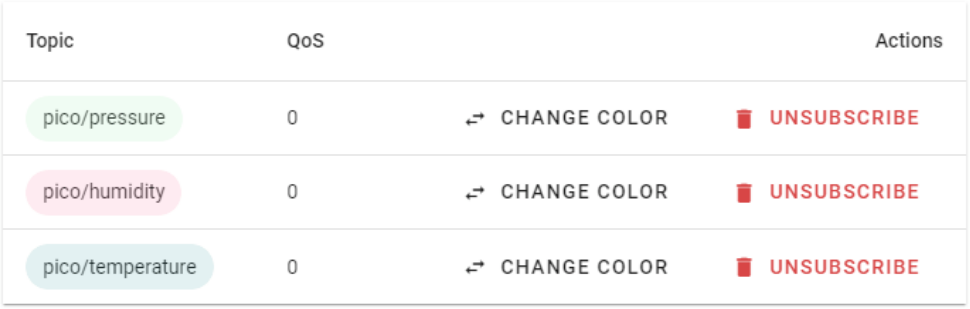
Go to your HiveMQ cluster, and click on the **Web Client** tab. Insert your MQTT broker username and password and click **Connect Client**.



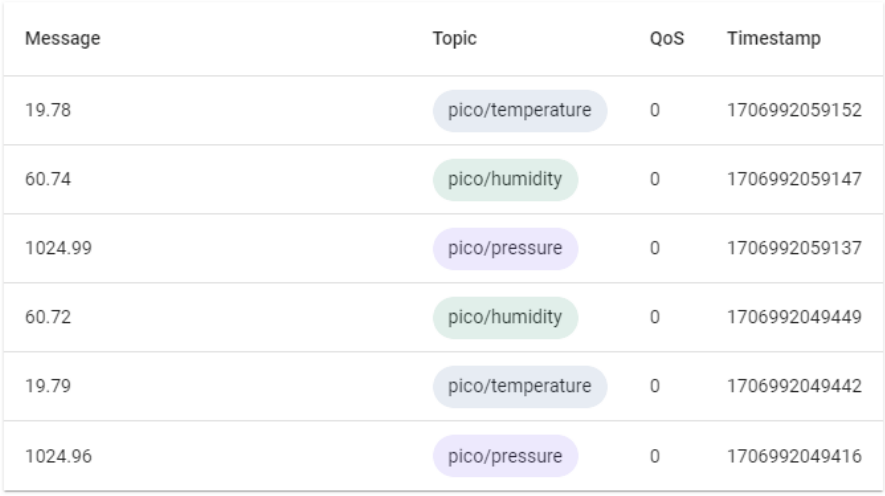
Subscribe to the topics that the Raspberry Pi Pico is publishing to. Fill in pico/temperature and then click on **+Subscribe**.



Repeat the process for `pico/humidity` and `pico/pressure`.



After subscribing to those topics, scroll down to the bottom of the page. You should start receiving the Raspberry Pi Pico MQTT messages.



And that's it. You successfully published MQTT messages with the Raspberry Pi Pico. Now, you can use any other MQTT Client to subscribe to those messages and receive the data. For example, having Node-RED nodes or Adafruit IO widgets subscribed to MQTT topics and displaying the data on charts and gauges.

You can also have any other microcontroller subscribed to those topics to receive the data—it can be another Raspberry Pi Pico, an ESP32, an ESP8266, or another board.

7.3 Key Code Points

Payload Must Be Bytes

```
# String to bytes — two approaches:
temperature = '25.23C'
client.publish(b'pico/temperature', temperature.encode())  # .encode() method

# For numeric values:
temp_float = 25.23
client.publish(b'pico/temperature', str(temp_float).encode())

# JSON payload (best practice for structured data):
import json
data = {'temperature': 25.23, 'humidity': 64.8, 'pressure': 1013.2}
client.publish(b'pico/sensors', json.dumps(data).encode(), retain=True)
```

retain=True Explained

With retain=True, the broker stores this as the last known value. Any new subscriber connecting later will immediately receive it — even if the Pico W published 10 minutes ago and is currently sleeping. Essential for sensor data where subscribers may connect at any time.

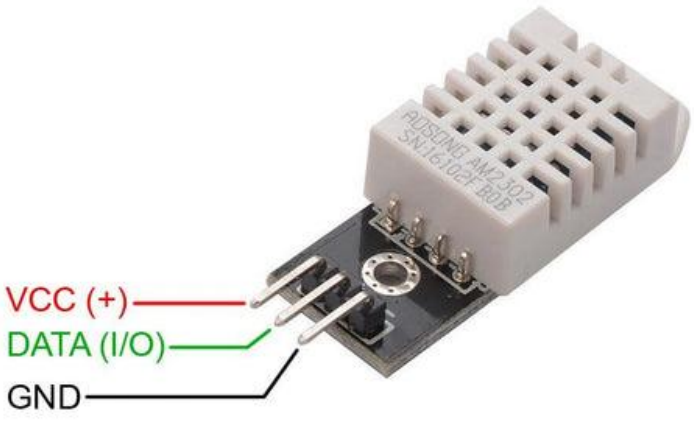
SSL for HiveMQ

HiveMQ Cloud requires SSL/TLS on port 8883. The ssl=True and ssl_params={'server_hostname': config.mqtt_server} parameters enable encrypted connection. For a local Mosquitto broker without SSL, simply omit both parameters (ssl defaults to False).

7.4 Testing with MQTT Tools

Tool	Platform	How to Use
MQTT Explorer	Windows/macOS/Linux	GUI browser for all topics. Shows live messages. Download: mqtt-explorer.com
HiveMQ Web Client	Any browser	Available inside your HiveMQ cluster dashboard under Web Client.
mosquitto_sub	Linux terminal	mosquitto_sub -h broker_ip -t 'pico/#' — shows all pico/ messages
Node-RED	Any	Drag MQTT input node, set broker/topic, connect to debug node.

7. Project A2 — Publishing MQTT Messages (DHT22 Sensor)



To replace the **BME280** with a **DHT22** sensor, you only need to make a few modifications:

Changes Required

BME280	DHT22
Uses I2C (SDA/SCL)	Uses single GPIO pin
Measures Temperature, Humidity, Pressure	Measures Temperature and Humidity only
Requires BME280.py library	Requires dht.py library
Publishes Temperature, Humidity, Pressure	Publishes Temperature and Humidity

Hardware Connection

Raspberry Pi Pico W ↔ DHT22

DHT22	Pico W
VCC	3.3V
DATA	GP15
GND	GND

Add a **10kΩ pull-up resistor** between VCC and DATA.

Revised MQTT Publisher Code (DHT22)

```
from machine import Pin
from time import sleep
import network
from umqtt.simple import MQTTClient
import config
import dht

# MQTT Topics
MQTT_TOPIC_TEMPERATURE = 'pico/temperature'
MQTT_TOPIC_HUMIDITY = 'pico/humidity'

# MQTT Parameters
MQTT_SERVER = config.mqtt_server
MQTT_PORT = 0
MQTT_USER = config.mqtt_username
MQTT_PASSWORD = config.mqtt_password

MQTT_CLIENT_ID = b"raspberrypi_picow"
MQTT_KEEPALIVE = 7200

MQTT_SSL = True
MQTT_SSL_PARAMS = {'server_hostname': MQTT_SERVER}

# Initialize DHT22
dht_sensor = dht.DHT22(Pin(15))


def get_sensor_readings():
    try:
        dht_sensor.measure()

        temperature = dht_sensor.temperature()
        humidity = dht_sensor.humidity()

        return temperature, humidity

    except Exception as e:
        print("DHT22 Error:", e)
        return None, None


def initialize_wifi(ssid, password):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    wlan.connect(ssid, password)

    timeout = 10

    while timeout > 0:
        if wlan.status() >= 3:
            break
```

```
        timeout -= 1
        print("Waiting for WiFi...")
        sleep(1)

    if wlan.status() != 3:
        return False

    print("WiFi Connected")
    print("IP Address:", wlan.ifconfig()[0])

    return True


def connect_mqtt():
    try:
        client = MQTTClient(
            client_id=MQTT_CLIENT_ID,
            server=MQTT_SERVER,
            port=MQTT_PORT,
            user=MQTT_USER,
            password=MQTT_PASSWORD,
            keepalive=MQTT_KEEPALIVE,
            ssl=MQTT_SSL,
            ssl_params=MQTT_SSL_PARAMS
        )

        client.connect()

        print("MQTT Connected")

        return client

    except Exception as e:
        print("MQTT Connection Error:", e)
        raise


def publish_mqtt(topic, value):
    client.publish(topic, str(value))
    print("Topic:", topic)
    print("Value:", value)
    print("Published Successfully")


try:

    if not initialize_wifi(
        config.wifi_ssid,
        config.wifi_password
    ):
        print("WiFi Connection Failed")

    else:

        client = connect_mqtt()
```

```
        while True:

            temperature, humidity = get_sensor_readings()

            if temperature is not None:

                publish_mqtt(
                    MQTT_TOPIC_TEMPERATURE,
                    temperature
                )

                publish_mqtt(
                    MQTT_TOPIC_HUMIDITY,
                    humidity
                )

            sleep(10)

except Exception as e:
    print("Error:", e)
```

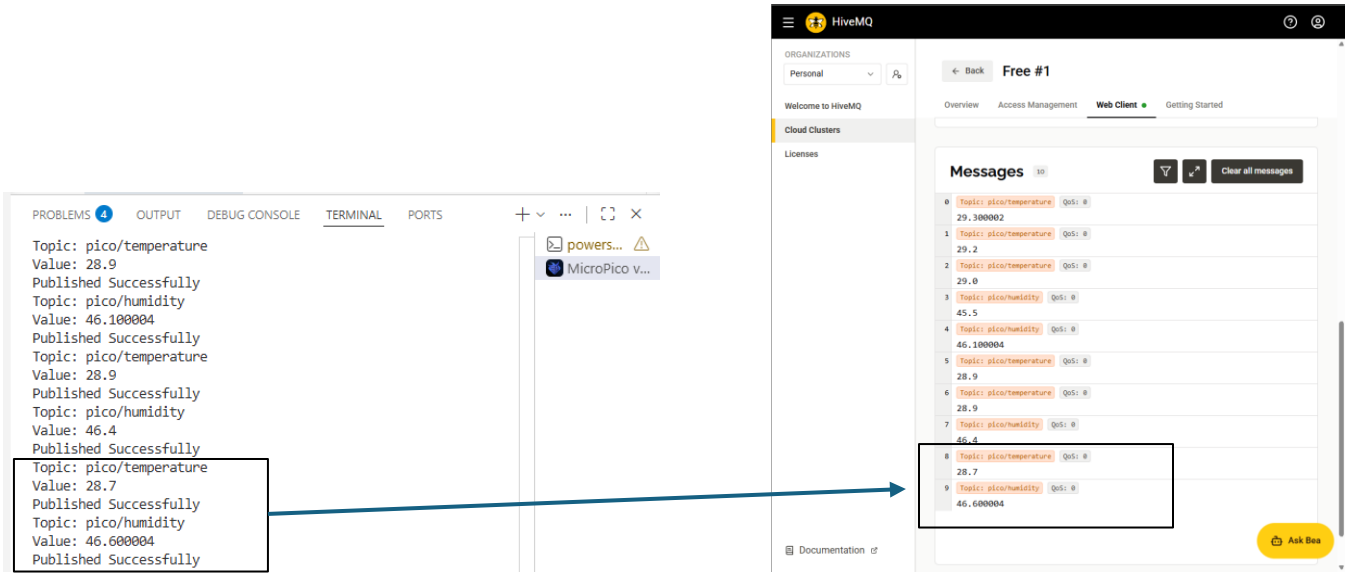
MQTT Topics Published

Topic	Example Payload
pico/temperature	29.4
pico/humidity	67.2

Example:

```
Topic:                                                  pico/temperature
Payload:                                                            29.4

Topic:                                                        pico/humidity
Payload: 67.2
```



If You Want to Keep the Pressure Topic

Since the DHT22 cannot measure pressure, you can either:

Option 1: Remove Pressure Completely
Delete:

```
MQTT_TOPIC_PRESSURE = 'pico/pressure'
```

and

```
publish_mqtt(MQTT_TOPIC_PRESSURE, str(pressure))
```

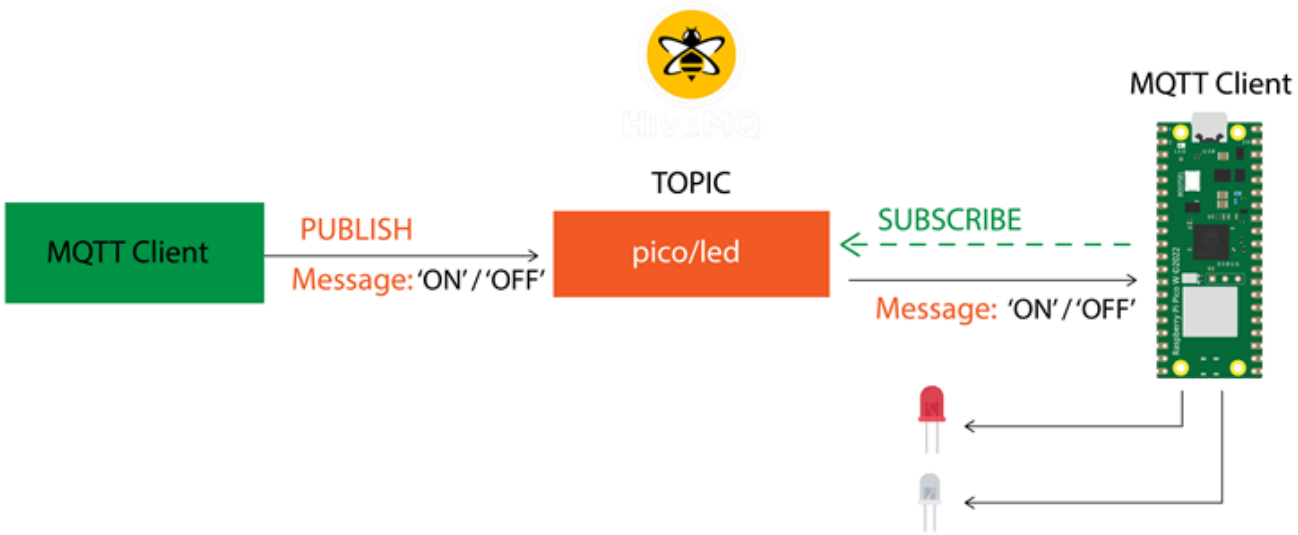
Option 2: Publish a Placeholder Value

```
MQTT_TOPIC_PRESSURE = 'pico/pressure'

publish_mqtt(MQTT_TOPIC_PRESSURE, "N/A")
```

8. Project B — Subscribing to MQTT Topics (LED Control)

The Pico W subscribes to `pico/led` and responds by turning the onboard LED on or off based on incoming messages. This is the receive side of MQTT — the Pico acts as a command consumer, accepting instructions from any MQTT client on the network.



8.1 Hardware

The onboard LED (Pin 'LED') requires no extra hardware. For an external LED: GPIO 15 via 220 Ohm resistor to GND.

Here's the workflow to subscribe to MQTT topics.

1. Connect the Raspberry Pi Pico to the internet;
2. Connect to the MQTT broker;
3. Subscribe to the MQTT topic;
4. Create and assign a callback function that will run when a message is received;
5. Create a loop that is constantly checking for new MQTT messages.

The following code subscribes to the pico/led topic and controls an LED according to the received message.

8.2 Complete Subscriber Script

```
from machine import Pin
from time import sleep
import network
from umqtt.simple import MQTTClient
import config

# Define LED
led = Pin('LED', Pin.OUT)
```

```
# Constants for MQTT Topics
MQTT_TOPIC_LED = 'pico/led'

# MQTT Parameters
MQTT_SERVER = config.mqtt_server
MQTT_PORT = 0
MQTT_USER = config.mqtt_username
MQTT_PASSWORD = config.mqtt_password
MQTT_CLIENT_ID = b'raspberrypi_picow'
MQTT_KEEPALIVE = 7200
MQTT_SSL = True   # set to False if using local Mosquitto MQTT broker
MQTT_SSL_PARAMS = {'server_hostname': MQTT_SERVER}


# Init Wi-Fi Interface
def initialize_wifi(ssid, password):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    # Connect to the network
    wlan.connect(ssid, password)

    # Wait for Wi-Fi connection
    connection_timeout = 10
    while connection_timeout > 0:
        if wlan.status() >= 3:
            break
        connection_timeout -= 1
        print('Waiting for Wi-Fi connection...')
        sleep(1)

    # Check if connection is successful
    if wlan.status() != 3:
        return False
    else:
        print('Connection successful!')
        network_info = wlan.ifconfig()
        print('IP address:', network_info[0])
        return True

# Connect to MQTT Broker
def connect_mqtt():
    try:
        client = MQTTClient(client_id=MQTT_CLIENT_ID,
                            server=MQTT_SERVER,
                            port=MQTT_PORT,
                            user=MQTT_USER,
                            password=MQTT_PASSWORD,
                            keepalive=MQTT_KEEPALIVE,
                            ssl=MQTT_SSL,
                            ssl_params=MQTT_SSL_PARAMS)
        client.connect()
        return client
    except Exception as e:
```

```
        print('Error connecting to MQTT:', e)

# Subcribe to MQTT topics
def subscribe(client, topic):
    client.subscribe(topic)
    print('Subscribe to topic:', topic)

# Callback function that runs when you receive a message on subscribed topic
def my_callback(topic, message):
    # Perform desired actions based on the subscribed topic and response
    print('Received message on topic:', topic)
    print('Response:', message)
    # Check the content of the received message
    if message == b'ON':
        print('Turning LED ON')
        led.value(1)  # Turn LED ON
    elif message == b'OFF':
        print('Turning LED OFF')
        led.value(0)  # Turn LED OFF
    else:
        print('Unknown command')

try:
    # Initialize Wi-Fi
    if not initialize_wifi(config.wifi_ssid, config.wifi_password):
        print('Error connecting to the network... exiting program')
    else:
        # Connect to MQTT broker, start MQTT client
        client = connect_mqtt()
        client.set_callback(my_callback)
        subscribe(client, MQTT_TOPIC_LED)

        # Continuously checking for messages
        while True:
            sleep(5)
            client.check_msg()
            print('Loop running')
except Exception as e:
    print('Error:', e)
```

Let's take a quick look at the relevant sections of code for this example.

If you're using a local Mosquitto MQTT broker, you should set the `MQTT_SSL` parameter to `False`.

```
MQTT_SSL = False
```

Subscribe to MQTT Topics

We create a function called `subscribe()` that accepts as arguments the MQTT client and the topic we want to subscribe to.

```
def subscribe(client, topic):
    client.subscribe(topic)
    print('Subscribe to topic:', topic)
```

Callback Function

We need to create a callback function that will run when a new message is received on a topic we are subscribed to. The topic and message arguments are automatically passed to the function when it is called.

```
def my_callback(topic, message):
    # Perform desired actions based on the subscribed topic and response
    print('Received message on topic:', topic)
    print('Response:', message)
    # Check the content of the received message
    if message == b'ON':
        print('Turning LED ON')
        led.value(1)  # Turn LED ON
    elif message == b'OFF':
        print('Turning LED OFF')
        led.value(0)  # Turn LED OFF
    else:
        print('Unknown command')
```

In this function, we check the content of the message. If it is `ON`, we'll turn the Raspberry Pi Pico built-in LED ON.

```
if message == b'ON':
    print('Turning LED ON')
    led.value(1)  # Turn LED ON
```

If the message is `OFF`, it will turn the LED off.

```
elif message == b'OFF':
    print('Turning LED OFF')
    led.value(0)  # Turn LED OFF
```

If we receive any other messages, we print `Unknown command`.

```
else:
    print('Unknown command')
```

Connecting to Wi-Fi, MQTT Broker and Checking Messages

To subscribe to the MQTT broker and constantly check for incoming messages, first we need to connect to the internet.

The following lines try to connect to the internet:

```
try:
    # Initialize Wi-Fi
    if not initialize_wifi(config.wifi_ssid, config.wifi_password):
        print('Error connecting to the network... exiting program')
```

If we succeed, we also connect to the MQTT broker.

```
client = connect_mqtt()
```

Then, assign the callback function by using the `set_callback()` and passing as argument the callback function we created previously `my_callback`.

```
client.set_callback(my_callback)
```

Finally, we can subscribe to MQTT topics. In this case, we're just subscribing to the `MQTT_TOPIC_LED`, but you can subscribe to multiple topics.

```
subscribe(client, MQTT_TOPIC_LED)
```

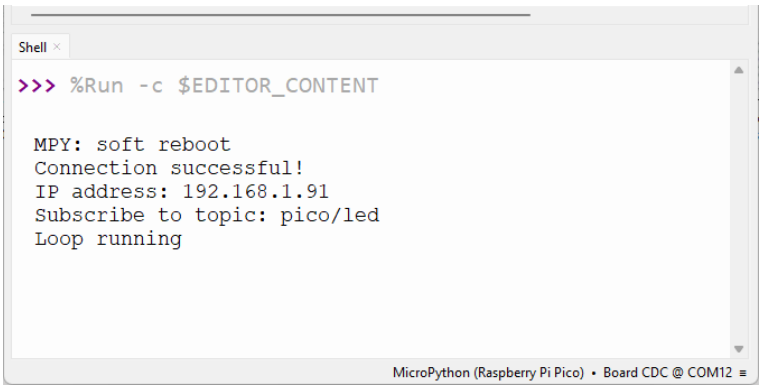
Then, create a while loop to constantly check for incoming messages using the `check_msg()` method.

```
while True:
    sleep(5)
    client.check_msg()
    print('Loop running')
```

You can also add other tasks to the loop. We check for new messages every five seconds, but you can use a smaller or longer delay time depending on how often you're expecting to receive messages.

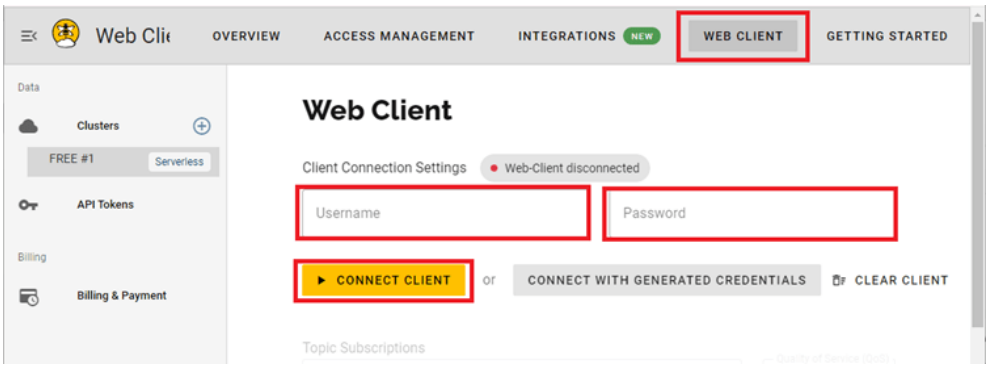
Testing the Code

Run the previous code on your Raspberry Pi Pico. It will connect to the internet and subscribe to the MQTT topic.



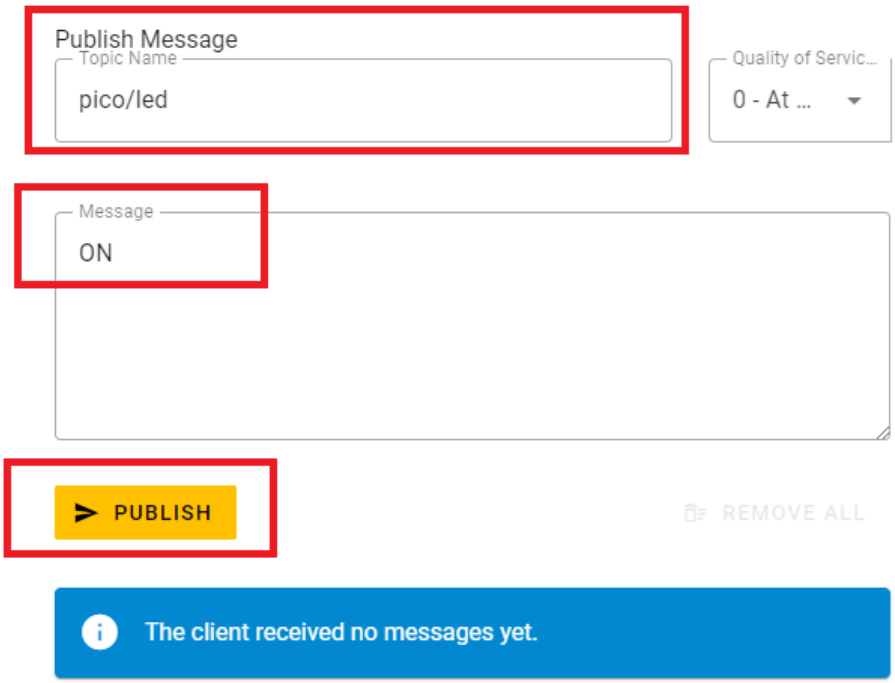
Now, let's publish some messages on the `pico/led` topic to control the LED using HiveMQ Web Client.

Go to your HiveMQ cluster Web Client. Insert your MQTT broker username and password and click **Connect Client**.



Scroll down the page to the *Publish Message* section.

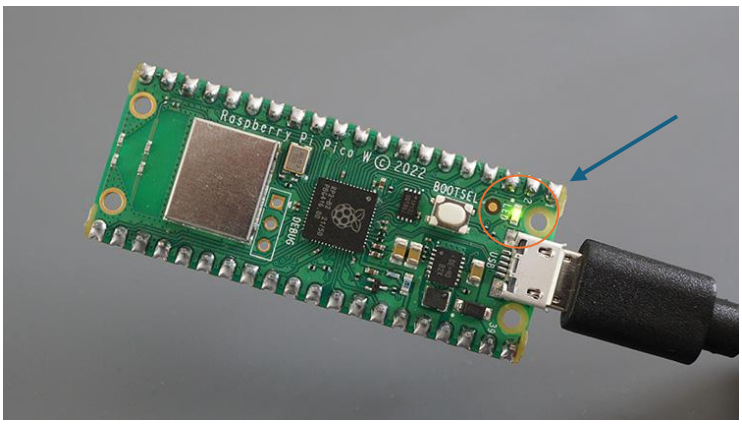
Insert `pico/led` on the **Topic Name** field, and type **ON** in the **Message** field.



Finally, press the **Publish** button.

Your Raspberry Pi Pico will receive the message, and will turn the on-board LED on.





Go back to HiveMQ Client and send an OFF message.



The Raspberry Pi Pico will receive the message and turn off the LED.



And that's it. You successfully subscribed to an MQTT topic and executed a different task according to the received message. Now, instead of HiveMQ Web Client, you can use any Home Automation or IoT platform with MQTT support that allows you to build a web interface with buttons or switches to send MQTT messages.

8.3 Critical Subscription Rules

Rule 1 — Set Callback Before Subscribe

```
# WRONG — subscribe before callback:
client.subscribe(b'pico/led')    # Error: callback not set
client.set_callback(callback)

# CORRECT — callback first, then subscribe:
client.set_callback(callback)
client.connect()
client.subscribe(b'pico/led')    # Now callback is ready to handle messages
```

Rule 2 — Compare Bytes, Not Strings

```
# WRONG — comparing bytes to string:
if msg == 'on':    # Always False — msg is bytes, 'on' is str

# CORRECT — compare bytes to bytes:
if msg == b'on':   # Correct

# OR decode first:
if msg.decode() == 'on':   # Also correct

# OR decode and normalise case:
if msg.decode().lower() == 'on':   # Handles 'ON', 'On', 'on'
```

check_msg() vs wait_msg()

Method	Behaviour	Use When
check_msg()	Non-blocking. Returns immediately if no message waiting.	You need to do other work between checks (sensors, display, etc.)
wait_msg()	Blocking. Halts execution until a message arrives.	Device only needs to react to messages, nothing else running.

8.4 Sending Test Commands

```
# Test from Linux terminal:
mosquitto_pub -h broker_ip -t 'pico/led' -m 'on'
mosquitto_pub -h broker_ip -t 'pico/led' -m 'off'
mosquitto_pub -h broker_ip -t 'pico/led' -m 'toggle'

# With authentication:
mosquitto_pub -h broker_ip -u username -P password -t 'pico/led' -m 'on'
```

9. Advanced MQTT Features

9.1 umqtt.robust — Automatic Reconnection

The umqtt.robust module is a drop-in replacement for umqtt.simple that adds automatic reconnection. If the network drops, robust catches the OSError, waits, and reconnects. Recommended for all deployed (unattended) devices.

```
# Just change the import — everything else is identical:
from umqtt.robust import MQTTClient   # instead of umqtt.simple

# robust automatically retries publish() and wait_msg() on failure.
# check_msg() accepts an 'attempts' parameter:
client.check_msg(attempts=3)   # Retry up to 3 times
```

9.2 Retained Messages

```
# Publish with retain — broker remembers this value:
client.publish(b'pico/temperature', b'25.3', retain=True)

# New subscriber connects 5 minutes later ->
# Immediately receives b'25.3' even though Pico W is sleeping.

# Clear retained message — publish empty payload:
client.publish(b'pico/temperature', b'', retain=True)
```

9.3 Last Will and Testament (LWT)

LWT lets the broker send a message on behalf of a client that disconnects abnormally (power loss, crash). Register the LWT before connecting.

```
# Set LWT BEFORE client.connect():
client.set_last_will(
    topic  = b'pico/status',
    msg    = b'offline',
    retain = True,   # Keep 'offline' until device reconnects
    qos    = 1
)
client.connect()

# Publish 'online' immediately after successful connect:
client.publish(b'pico/status', b'online', retain=True)

# Result: pico/status shows 'online' while running,
# automatically becomes 'offline' if device unexpectedly drops.
```

9.4 Multiple Topic Subscriptions & Wildcards

```
# Subscribe to multiple specific topics:
client.set_callback(multi_callback)
client.connect()
client.subscribe(b'pico/led')
client.subscribe(b'pico/buzzer')

# Or subscribe to all pico/ subtopics with wildcard:
client.subscribe(b'pico/#')

# Differentiate topics in callback:
def multi_callback(topic, msg):
    if topic == b'pico/led':
        handle_led(msg)
    elif topic == b'pico/buzzer':
        handle_buzzer(msg)
```

9.5 JSON Payloads

```
import json

# Publish structured data as JSON:
data = {'temperature': 25.3, 'humidity': 64.8, 'pressure': 1013.2}
client.publish(b'pico/sensors', json.dumps(data).encode(), retain=True)

# Subscriber — decode JSON:
def callback(topic, msg):
    if topic == b'pico/sensors':
        readings = json.loads(msg.decode())
        temp = readings['temperature']
        print(f'Temperature: {temp}C')
```

10. MQTT in Real IoT Systems

10.1 Integration Platforms

Platform	Role	MQTT Integration	Free Tier
Node-RED	Visual flow programming / dashboard	Built-in MQTT in/out nodes	Yes (self-hosted)
Home Assistant	Home automation hub	Native MQTT integration	Yes (self-hosted)
Grafana	Data visualisation	Via InfluxDB or MQTT plugin	Yes (self-hosted)
Adafruit IO	Cloud IoT dashboard & storage	MQTT feed topics built-in	Yes (limited)
AWS IoT Core	Enterprise cloud IoT	MQTT as primary protocol	12 months free tier

10.2 Node-RED Flow — Receive Pico W Sensor Data

```
# Node-RED receives pico/temperature and displays on a gauge widget

# Flow: [MQTT in: pico/temperature] -> [Function node] -> [Gauge widget]

# Function node JavaScript to parse value:
# var raw = msg.payload;          // e.g. '25.23C'
# msg.payload = parseFloat(raw);  // → 25.23
# return msg;

# To send LED command from Node-RED button:
# [inject node: payload='on'] -> [MQTT out: pico/led]
```

10.3 Home Assistant YAML Configuration

```
# configuration.yaml — MQTT sensors and switch for Pico W
mqtt:
  sensor:
    - name: 'Pico Temperature'
      state_topic: 'pico/temperature'
      unit_of_measurement: 'C'
      device_class: temperature
    - name: 'Pico Humidity'
      state_topic: 'pico/humidity'
      unit_of_measurement: '%'
      device_class: humidity
  switch:
```

```
    - name: 'Pico LED'
      command_topic: 'pico/led'
      payload_on:  'on'
      payload_off: 'off'
      state_topic: 'pico/status'
```

10.4 Complete System Architecture

```
#  ╔═══════════════════════════════════════════════════════════╗
#  ║              Complete IoT MQTT Architecture               ║
#  ╚═══════════════════════════════════════════════════════════╝
#
#  [Pico W + BME280]          [Pico W + LED]          [Mobile App]
#  PUBLISH:                   SUBSCRIBE:               PUBLISH:
#  pico/temperature           pico/led                 pico/led
#  pico/humidity
#  pico/pressure
#         |                        ▲                      |
#         └──────────────────┐     |    ──────────────────┘
#                            ▼     |
#             ┌──────────────────────────────────────┐
#             |           MQTT BROKER                |
#             |  (HiveMQ Cloud / Mosquitto on RPi)   |
#             └──────────────────────────────────────┘
#                  |               |             |
#                  ▼               ▼             ▼
#           [Node-RED]       [Home Assistant]  [MQTT Explorer]
#           Dashboard        Automations       Debug / Testing
```

11. Featured Resource & Further Reading

FEATURED RESOURCE — Highly Recommended

Random Nerd Tutorials: Raspberry Pi Pico W — Getting Started with MQTT (MicroPython)

URL: https://randomnerdtutorials.com/raspberry-pi-pico-w-mqtt-micropython/

This is the primary source for this lecture and one of the most comprehensive beginner-friendly guides to MQTT on the Raspberry Pi Pico W with MicroPython. Written by Sara Santos and Rui Santos, it covers the complete workflow from first principles: introducing MQTT concepts clearly, setting up HiveMQ as a cloud broker with step-by-step screenshots, installing umqtt modules through Thonny IDE, creating the config.py credentials file, and providing two fully tested projects — one for publishing BME280 sensor readings and one for subscribing to control an LED. The tutorial includes real Thonny IDE output screenshots and a working demonstration, making it an invaluable follow-along reference for students new to MQTT and IoT communication.

Why it stands out: Complete beginner-to-working-project coverage • Step-by-step screenshots • Two real hardware projects (publish + subscribe) • HiveMQ cloud setup guide • Actively maintained • Part of the full Pico W MicroPython eBook series

11.1 Continuation Tutorials

Tutorial	URL	What It Adds
MQTT with Node-RED	randomnerdtutorials.com/esp32-mqtt-publish-subscribe-arduino-ide/	Visualise sensor data in a Node-RED dashboard
Mosquitto on Raspberry Pi	randomnerdtutorials.com/how-to-install-mosquitto-broker-on-raspberry-pi/	Install and configure local broker
Cloud MQTT Broker	randomnerdtutorials.com/cloud-mqtt-mosquitto-broker-access-anywhere-digital-ocean/	Deploy Mosquitto on a cloud VPS
BME280 Deep Dive	randomnerdtutorials.com/raspberry-pi-pico-bme280-micropython/	BME280 library installation and I2C usage
Home Automation	randomnerdtutorials.com/home-automation-using-raspberry-pi/	Control Pico W from Home Assistant automations

11.2 Official References

- MQTT 5.0 Specification — docs.oasis-open.org/mqtt/mqtt/v5.0/
- MicroPython umqtt source — github.com/micropython/micropython-lib/tree/master/micropython/umqtt.simple
- HiveMQ Documentation — docs.hivemq.com
- Mosquitto Documentation — mosquitto.org/documentation/
- MQTT Explorer GUI tool — mqtt-explorer.com
- Node-RED MQTT nodes — nodered.org
- Home Assistant MQTT — home-assistant.io/integrations/mqtt/

12. Summary, Review Questions & Lab Exercises

12.1 Key Concepts Summary

Concept	Key Point
MQTT	Lightweight pub/sub protocol. 2-byte minimum packet. Persistent TCP connection.
Publish	Send message to a topic. Broker routes to all subscribers. Payload = bytes.
Subscribe	Register interest in topic. callback() called on each arrival.
Topic	Hierarchical bytes string. Case-sensitive. / separator. + and # wildcards.
Broker	Central hub: HiveMQ (cloud) or Mosquitto (local). Routes and stores messages.
QoS 0	At most once. No guarantee. Lowest overhead. Use for frequent sensor data.
QoS 1	At least once. Guaranteed, possible duplicates. Use for control commands.
QoS 2	Exactly once. Highest guarantee and overhead. Use for critical operations.
umqtt.simple	Core MQTT client module. connect(), publish(), subscribe(), check_msg().
umqtt.robust	Extends simple with auto-reconnect. Use for deployed unattended devices.
config.py	Separate credentials file. MQTT values must be bytes (b prefix).
Retain	retain=True: broker stores last message, new subscribers receive it immediately.
Last Will	Pre-registered message sent by broker if client disconnects abnormally.
check_msg()	Non-blocking poll. Returns immediately. Use when doing other tasks in loop.
Bytes payload	.encode() converts string to bytes. .decode() converts bytes to string.

12.2 Review Questions

A — MQTT Concepts

1. What does MQTT stand for? Describe its original use case and why it suits IoT.
2. Explain publish/subscribe. How does it differ from HTTP request-response?
3. A Pico W publishes to pico/temperature. Which device receives it: (a) all MQTT devices, (b) only devices subscribed to pico/temperature, (c) devices subscribed to pico/#? Explain.
4. What are topics? Why are they case-sensitive? Design a topic hierarchy for a 3-room smart home with temperature, humidity, and lamp control.
5. What is the broker's role? Compare HiveMQ Cloud and Mosquitto on Raspberry Pi.
6. Describe QoS 0, 1, and 2. State one appropriate use case for each level.

B — MicroPython Implementation

1. What two umqtt modules are used and what is the key difference between them?
2. Why must MQTT server, username, and password be bytes objects (b'...') in umqtt?

3. Write code to create and connect an MQTTClient to a local Mosquitto broker at 192.168.1.79 port 1883 with no username/password.
4. What must you do before calling client.subscribe()? Why will it fail if skipped?
5. Explain check_msg() vs wait_msg(). Which would you use if you also need to read a sensor every second?
6. Write the callback function that receives topic=b'pico/led' and msg=b'ON' and turns on the onboard LED.

C — Advanced & Systems

1. What does retain=True do on publish()? Describe a scenario where this is essential.
2. Write code to configure an LWT that publishes b'offline' to b'pico/status' with retain=True when the Pico W disconnects unexpectedly.
3. How would you publish temperature=25.3 and humidity=64.8 as a single JSON payload to b'pico/sensors'?
4. A subscriber script stops receiving messages after several hours of operation. List two possible causes and their fixes.

12.3 Lab Exercises

Lab 1 — Publisher with Random Values (Beginner, 30 min)

1. Install umqtt.simple and umqtt.robust on your Pico W via Thonny IDE.
2. Create config.py with your Wi-Fi and HiveMQ (or Mosquitto) credentials.
3. Write a script that generates random temperature (20–35), humidity (40–80), and pressure (990–1020) and publishes to pico/temperature, pico/humidity, pico/pressure every 5 seconds.
4. Open MQTT Explorer or HiveMQ Web Client and verify messages arrive every 5 seconds.
5. Test retain=True: subscribe after stopping the publisher and verify you still receive the last values.

Lab 2 — LED Subscriber (Beginner, 30 min)

1. Connect an LED to GPIO 15 with a 220 Ohm resistor.
2. Write the subscriber script from Section 8 and run it.
3. From MQTT Explorer or mosquitto_pub, send 'on', 'off', and 'toggle' to pico/led.
4. Verify the LED responds correctly. Modify the callback to handle both uppercase and lowercase variants.

Lab 3 — Bidirectional: Publish + Subscribe (Intermediate, 60 min)

1. Write a single script that both publishes temperature data every 10 seconds AND subscribes to pico/led for LED control.
2. Add an LWT: register b'offline' on b'pico/status' before connect(), then publish b'online' after connect().
3. Test simultaneously: send LED commands while the publish loop is running — both should operate independently.

Lab 4 — Node-RED Dashboard (Advanced, 90 min)

1. Install Node-RED on a computer (nodered.org/docs/getting-started/).
2. Wire a BME280 and run the full publisher from Section 7.
3. Create a Node-RED flow with 3 MQTT input nodes (one per sensor topic) feeding 3 Gauge dashboard widgets.
4. Add a Node-RED button that publishes 'toggle' to pico/led from the browser dashboard.

12.4 Quick Reference

Import	`from umqtt.simple import MQTTClient  OR  from umqtt.robust import MQTTClient`
Create	`client = MQTTClient(client_id, server, port, user, password, keepalive, ssl)`
Connect	`client.connect()`
Publish	`client.publish(b'topic', b'payload', retain=False, qos=0)`
Set callback	`client.set_callback(my_func)  — BEFORE subscribe()`
Subscribe	`client.subscribe(b'topic', qos=0)`
Check msg	`client.check_msg()  (non-blocking)  /  client.wait_msg()  (blocking)`
Last Will	`client.set_last_will(b'topic', b'msg', retain, qos)  — BEFORE connect()`
Ping	`client.ping()`
Disconnect	`client.disconnect()`
str to bytes	`'hello'.encode()  →  b'hello'`
bytes to str	`b'hello'.decode()  →  'hello'`
JSON publish	`json.dumps(dict).encode()`
JSON receive	`json.loads(bytes.decode())`
Unique client ID	`hexlify(machine.unique_id())  (from ubinascii import hexlify)`

simple.py

```
# forked from: https://github.com/micropython/micropython-lib/tree/master/micropython/umqtt.simple
import usocket as socket
import ustruct as struct
from ubinascii import hexlify


class MQTTException(Exception):
    pass


class MQTTClient:
    def __init__(
        self,
        client_id,
        server,
        port=0,
        user=None,
        password=None,
        keepalive=0,
        ssl=False,
        ssl_params={},
    ):
        if port == 0:
            port = 8883 if ssl else 1883
        self.client_id = client_id
        self.sock = None
        self.server = server
        self.port = port
        self.ssl = ssl
        self.ssl_params = ssl_params
        self.pid = 0
        self.cb = None
        self.user = user
        self.pswd = password
        self.keepalive = keepalive
        self.lw_topic = None
        self.lw_msg = None
        self.lw_qos = 0
        self.lw_retain = False

    def _send_str(self, s):
        self.sock.write(struct.pack("!H", len(s)))
        self.sock.write(s)
```

```
    def _recv_len(self):
        n = 0
        sh = 0
        while 1:
            b = self.sock.read(1)[0]
            n |= (b & 0x7F) << sh
            if not b & 0x80:
                return n
            sh += 7

    def set_callback(self, f):
        self.cb = f

    def set_last_will(self, topic, msg, retain=False, qos=0):
        assert 0 <= qos <= 2
        assert topic
        self.lw_topic = topic
        self.lw_msg = msg
        self.lw_qos = qos
        self.lw_retain = retain

    def connect(self, clean_session=True):
        self.sock = socket.socket()
        addr = socket.getaddrinfo(self.server, self.port)[0][-1]
        self.sock.connect(addr)
        if self.ssl:
            import ssl

            self.sock = ssl.wrap_socket(self.sock, **self.ssl_params)
        premsg = bytearray(b"\x10\0\0\0\0\0")
        msg = bytearray(b"\x04MQTT\x04\x02\0\0")

        sz = 10 + 2 + len(self.client_id)
        msg[6] = clean_session << 1
        if self.user is not None:
            sz += 2 + len(self.user) + 2 + len(self.pswd)
            msg[6] |= 0xC0
        if self.keepalive:
            assert self.keepalive < 65536
            msg[7] |= self.keepalive >> 8
            msg[8] |= self.keepalive & 0x00FF
        if self.lw_topic:
            sz += 2 + len(self.lw_topic) + 2 + len(self.lw_msg)
            msg[6] |= 0x4 | (self.lw_qos & 0x1) << 3 | (self.lw_qos & 0x2) << 3
            msg[6] |= self.lw_retain << 5

        i = 1
        while sz > 0x7F:
```

```
            premsg[i] = (sz & 0x7F) | 0x80
            sz >>= 7
            i += 1
        premsg[i] = sz

        self.sock.write(premsg, i + 2)
        self.sock.write(msg)
        # print(hex(len(msg)), hexlify(msg, ":"))
        self._send_str(self.client_id)
        if self.lw_topic:
            self._send_str(self.lw_topic)
            self._send_str(self.lw_msg)
        if self.user is not None:
            self._send_str(self.user)
            self._send_str(self.pswd)
        resp = self.sock.read(4)
        assert resp[0] == 0x20 and resp[1] == 0x02
        if resp[3] != 0:
            raise MQTTException(resp[3])
        return resp[2] & 1

    def disconnect(self):
        self.sock.write(b"\xe0\0")
        self.sock.close()

    def ping(self):
        self.sock.write(b"\xc0\0")

    def publish(self, topic, msg, retain=False, qos=0):
        pkt = bytearray(b"\x30\0\0\0")
        pkt[0] |= qos << 1 | retain
        sz = 2 + len(topic) + len(msg)
        if qos > 0:
            sz += 2
        assert sz < 2097152
        i = 1
        while sz > 0x7F:
            pkt[i] = (sz & 0x7F) | 0x80
            sz >>= 7
            i += 1
        pkt[i] = sz
        # print(hex(len(pkt)), hexlify(pkt, ":"))
        self.sock.write(pkt, i + 1)
        self._send_str(topic)
        if qos > 0:
            self.pid += 1
            pid = self.pid
            struct.pack_into("!H", pkt, 0, pid)
```

```
            self.sock.write(pkt, 2)
        self.sock.write(msg)
        if qos == 1:
            while 1:
                op = self.wait_msg()
                if op == 0x40:
                    sz = self.sock.read(1)
                    assert sz == b"\x02"
                    rcv_pid = self.sock.read(2)
                    rcv_pid = rcv_pid[0] << 8 | rcv_pid[1]
                    if pid == rcv_pid:
                        return
        elif qos == 2:
            assert 0

    def subscribe(self, topic, qos=0):
        assert self.cb is not None, "Subscribe callback is not set"
        pkt = bytearray(b"\x82\0\0\0")
        self.pid += 1
        struct.pack_into("!BH", pkt, 1, 2 + 2 + len(topic) + 1, self.pid)
        # print(hex(len(pkt)), hexlify(pkt, ":"))
        self.sock.write(pkt)
        self._send_str(topic)
        self.sock.write(qos.to_bytes(1, "little"))
        while 1:
            op = self.wait_msg()
            if op == 0x90:
                resp = self.sock.read(4)
                # print(resp)
                assert resp[1] == pkt[2] and resp[2] == pkt[3]
                if resp[3] == 0x80:
                    raise MQTTException(resp[3])
                return

    # Wait for a single incoming MQTT message and process it.
    # Subscribed messages are delivered to a callback previously
    # set by .set_callback() method. Other (internal) MQTT
    # messages processed internally.
    def wait_msg(self):
        res = self.sock.read(1)
        self.sock.setblocking(True)
        if res is None:
            return None
        if res == b"":
            raise OSError(-1)
        if res == b"\xd0":  # PINGRESP
            sz = self.sock.read(1)[0]
            assert sz == 0
```

```
            return None
        op = res[0]
        if op & 0xF0 != 0x30:
            return op
        sz = self._recv_len()
        topic_len = self.sock.read(2)
        topic_len = (topic_len[0] << 8) | topic_len[1]
        topic = self.sock.read(topic_len)
        sz -= topic_len + 2
        if op & 6:
            pid = self.sock.read(2)
            pid = pid[0] << 8 | pid[1]
            sz -= 2
        msg = self.sock.read(sz)
        self.cb(topic, msg)
        if op & 6 == 2:
            pkt = bytearray(b"\x40\x02\0\0")
            struct.pack_into("!H", pkt, 2, pid)
            self.sock.write(pkt)
        elif op & 6 == 4:
            assert 0
        return op

    # Checks whether a pending message from server is available.
    # If not, returns immediately with None. Otherwise, does
    # the same processing as wait_msg.
    def check_msg(self):
        self.sock.setblocking(False)
        return self.wait_msg()
```

robust.py

```
# forked from: https://github.com/micropython/micropython-lib/tree/master/micropython/umqtt.robust
import utime
from . import simple


class MQTTClient(simple.MQTTClient):
    DELAY = 2
    DEBUG = False

    def delay(self, i):
        utime.sleep(self.DELAY)

    def log(self, in_reconnect, e):
        if self.DEBUG:
            if in_reconnect:
                print("mqtt reconnect: %r" % e)
            else:
                print("mqtt: %r" % e)

    def reconnect(self):
        i = 0
        while 1:
            try:
                return super().connect(False)
            except OSError as e:
                self.log(True, e)
                i += 1
                self.delay(i)

    def publish(self, topic, msg, retain=False, qos=0):
        while 1:
            try:
                return super().publish(topic, msg, retain, qos)
            except OSError as e:
                self.log(False, e)
            self.reconnect()

    def wait_msg(self):
        while 1:
            try:
                return super().wait_msg()
            except OSError as e:
                self.log(False, e)
            self.reconnect()
```

```
    def check_msg(self, attempts=2):
        while attempts:
            self.sock.setblocking(False)
            try:
                return super().wait_msg()
            except OSError as e:
                self.log(False, e)
            self.reconnect()
            attempts -= 1
```